

High Availability & Elasticity

A practical introduction to Route 53, CloudFront and Elastic Load Balancing

Table of Contents

- 00 High Availability & Elasticity**
The ravel-agency problem and the design goals
- 01 Traffic, Delivery & Load Balancing**
Route 53, CloudFront and Elastic Load Balancing
- 02 Practice Lab**
Hands-on implementation - to be added

00 High Availability & Elasticity

Background

Our travel agency recently experienced a major outage during a promotional event.

- Increased web traffic overloaded the application and caused requests to time out.
- The EC2 instances were hosted in a single Availability Zone.
- Severe storms affected that Availability Zone, leaving the application offline for hours.
- The application needs to handle increases in traffic and remain available if one Availability Zone is affected.

Problems

The application currently depends too heavily on a fixed set of EC2 instances in one Availability Zone. We need to:

- Automatically adjust the number of EC2 instances when demand changes.
- Distribute instances across multiple Availability Zones.
- Distribute incoming traffic across the available instances.
- Detect unhealthy EC2 instances so traffic is not sent to them.

Solution

Use EC2 Auto Scaling together with a load balancer.

Create an Auto Scaling group across multiple Availability Zones. This distributes the EC2 instances instead of relying on a single location and allows the number of instances to automatically increase or decrease based on demand.

Then attach a load balancer to the Auto Scaling group. The load balancer distributes incoming traffic across the available EC2 instances.

The load balancer can also perform health checks to determine whether instances are healthy. If an instance becomes unhealthy, traffic can be directed to the remaining healthy instances.

No system is completely failure-proof, but Availability Zones are geographically separate. This makes a single event less likely to affect all of the application's instances at the same time.

Recap - High Availability and Elasticity

High availability means designing the application so it can continue operating when part of the infrastructure becomes unavailable. Running EC2 instances across multiple Availability Zones reduces reliance on a single location.

Elasticity means automatically adjusting resources as demand changes. EC2 Auto Scaling can add instances when traffic increases and remove unnecessary capacity when demand falls.

01 Traffic, Delivery & Load Balancing

Amazon Route 53, AWS's DNS Service

DNS is often compared to the internet's address book. Humans use names such as `example.com`, while computers need network addresses. Route 53 can take a domain name and route the request toward the appropriate AWS resource.

Route 53 is a highly available and scalable Domain Name System service.

It can be used to register domain names and create DNS records that determine where traffic for those domains should be directed.

Imagine you've built your portfolio and deployed it using AWS Amplify. AWS gives the application its own hosting address:

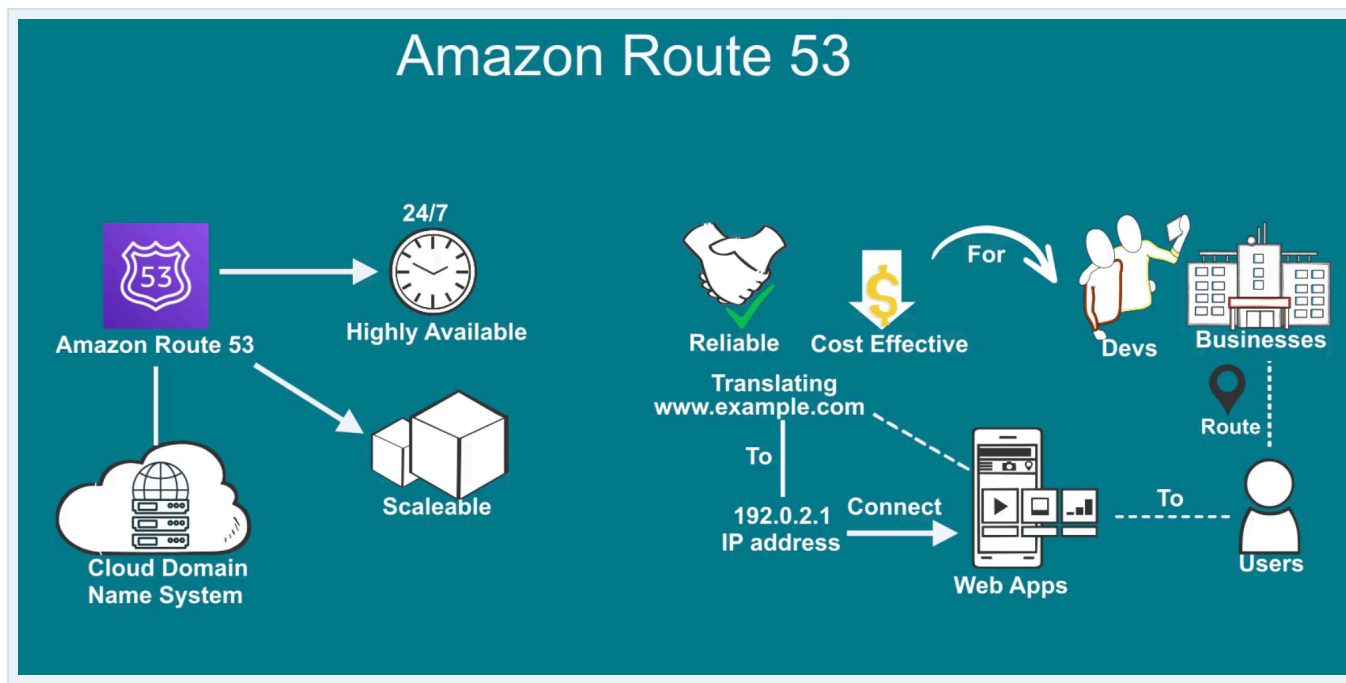
```
https://main.d123abc.amplifyapp.com
```

That's functional, but you want people to visit:

```
https://my-domain.com
```

You buy/register the domain and configure its DNS records so the domain points toward the service hosting your application.

Or perhaps you move your application somewhere else later. You update the DNS configuration, and the domain stays the same for your users even though the infrastructure behind it changed.



NOTE: Route 53 can direct a domain name to an Elastic Load Balancer. The load balancer then distributes the incoming traffic across healthy targets, such as EC2 instances in multiple Availability Zones.

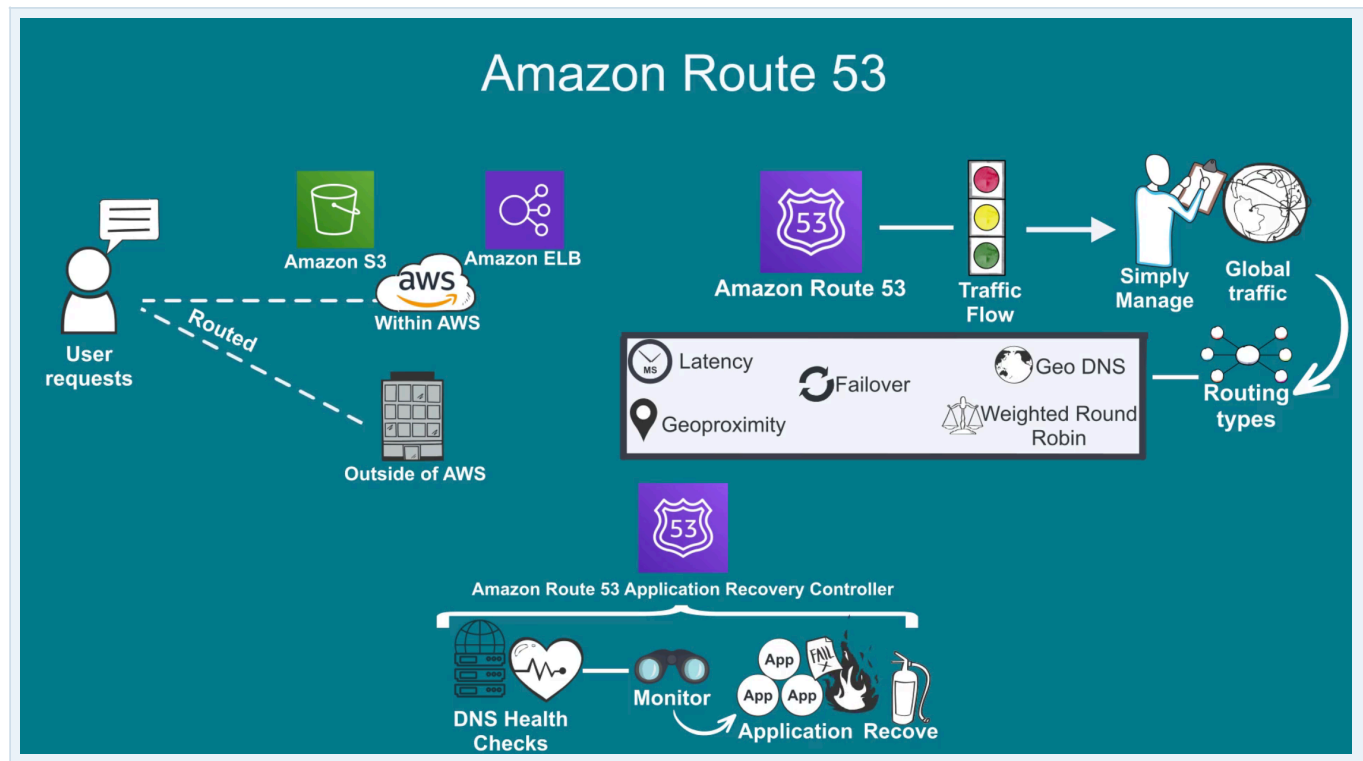
Route 53 Traffic Routing

Route 53 can route users to different resources depending on how you configure its routing policy. The destination can be inside AWS, such as an Elastic Load Balancer or S3, or even infrastructure outside AWS.

Some useful routing policies include:

- Latency-based: directs users toward the AWS Region that provides the lowest latency.
- Failover: directs traffic to a backup resource if the primary resource becomes unhealthy.
- Geolocation / geoproximity: routes traffic based on geographic location.
- Weighted: splits traffic between multiple resources according to configured percentages.

Route 53 can also perform health checks, allowing DNS routing decisions to take resource health into account.



NOTE: Route 53 doesn't decide on its own how traffic is routed. You configure a routing policy and its rules, and Route 53 then performs routing automatically according to those rules.

Route 53 Functionality

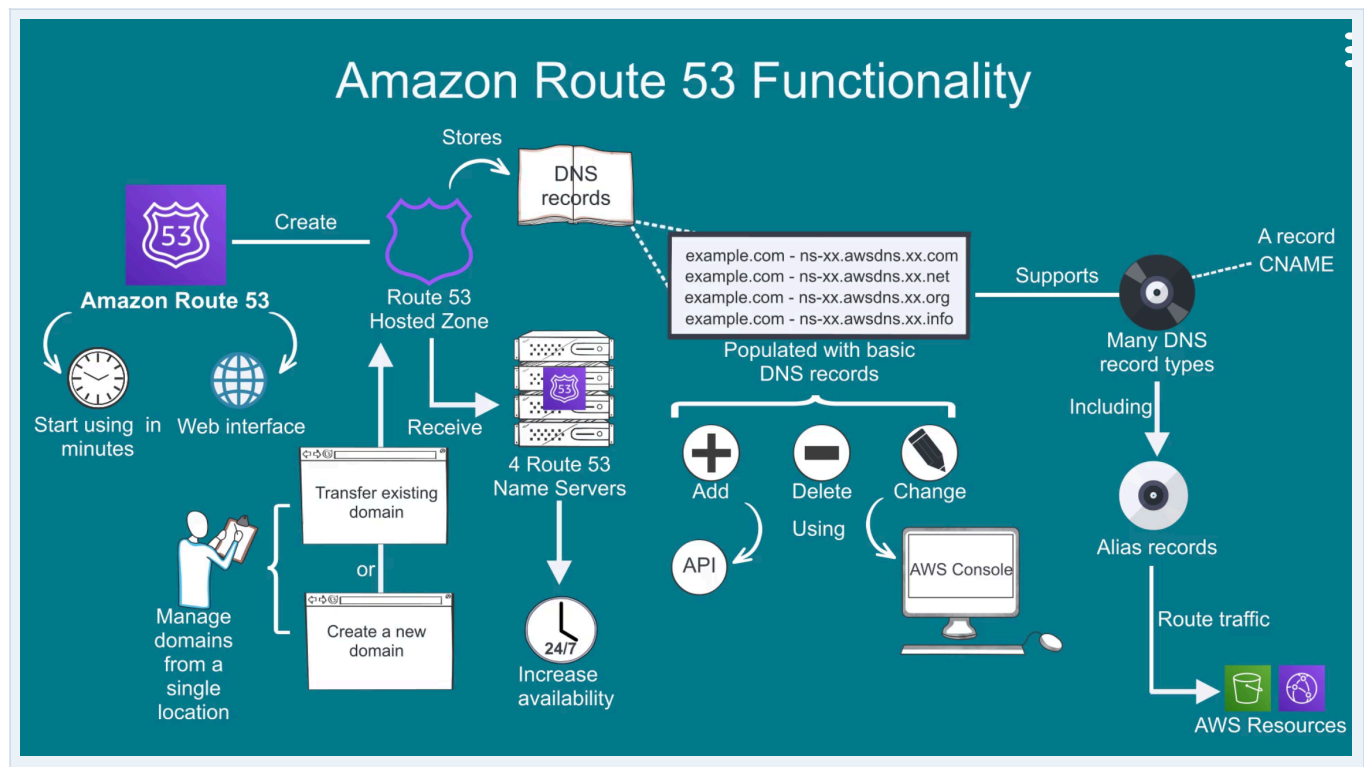
A hosted zone is the container where Route 53 stores the DNS records for a domain.

For example, if you own example.com, you could have DNS records such as:

- inspiira.se - your application
- www.inspiira.se - your application
- api.inspiira.se - your API

Route 53 supports standard DNS records such as A and CNAME records, as well as Alias records that can route traffic directly to supported AWS resources such as Elastic Load Balancers.

Record Type	What it does	Example
A	Points a domain to an IPv4 address	example.com > 192.0.2.1
CNAME	Points one domain name to another domain name	www.example.com > example.com
Alias	Points a domain to a supported AWS resource	example.com > Load Balancer



NOTE: Although CNAME records can reference an ELB's DNS name, Route 53 Alias records are generally preferred for Elastic Load Balancers. They can point directly to the ELB and can also be used for the root domain (example.com).

CloudFront

Route 53 can direct a user to our application, but what happens if that user is on the other side of the world?

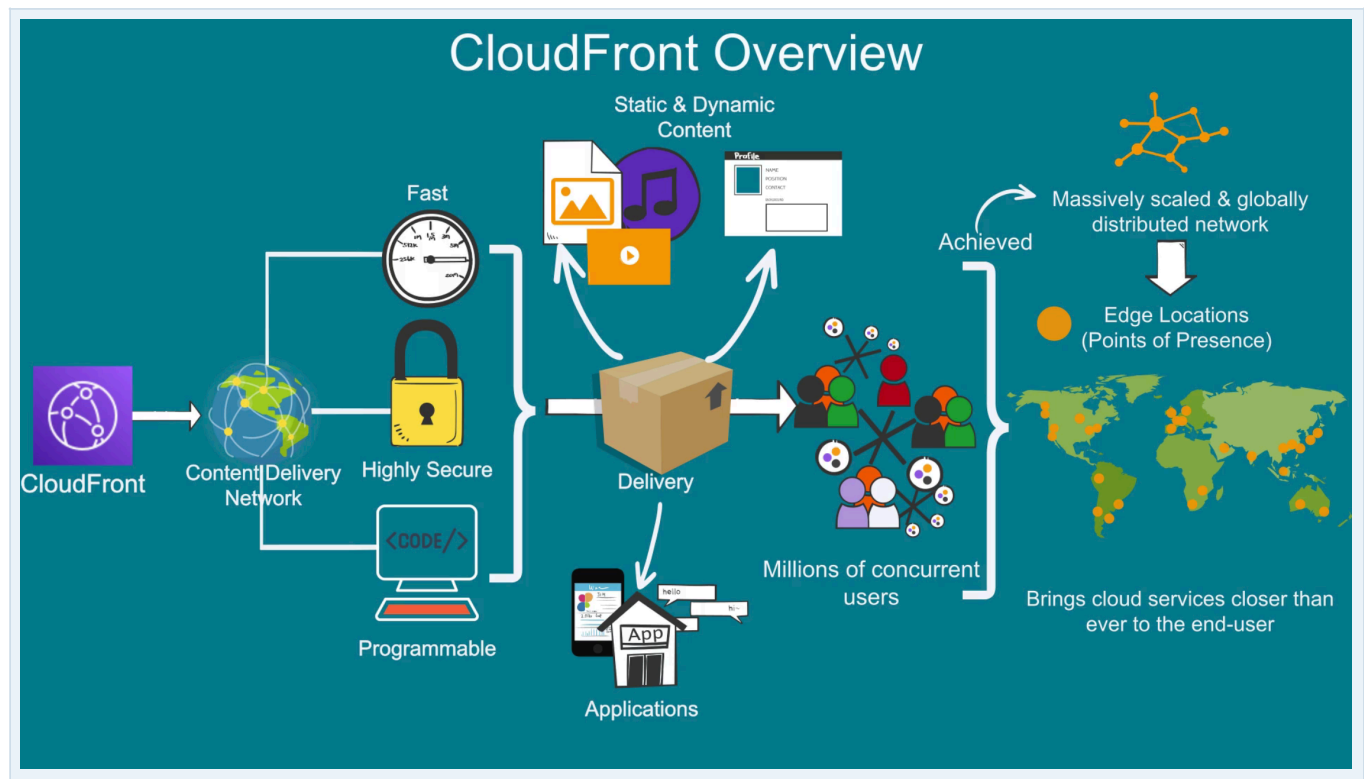
If our application is hosted in one AWS Region, users far away from that Region may experience higher latency because their requests have farther to travel.

This is where Amazon CloudFront comes in.

CloudFront is AWS's Content Delivery Network (CDN). It uses a global network of edge locations to deliver content closer to users. Frequently requested content can be cached at these locations instead of being retrieved from the original server every time.

The server or AWS resource where the original content comes from is called the origin. An origin could, for example, be an S3 bucket or an Elastic Load Balancer.

Service	Main responsibility	Example
Route 53	Directs users to the correct destination	example.com > CloudFront
CloudFront	Delivers content from locations closer to users	Cached images from a nearby edge location
Elastic Load Balancer	Distributes requests across healthy servers	Request > healthy EC2 instance
EC2 / S3	Hosts the original application or	Web application, images, files



Benefits

Fast Content Delivery

Why would CloudFront make my application faster?

CloudFront has Points of Presence (edge locations) around the world. Instead of every user retrieving content directly from your origin, CloudFront can serve cached content from a location closer to them.

This reduces latency, especially for users who are geographically far from the application's origin.

For example, if your application is hosted in Germany, a user in Japan may be able to retrieve cached images, JavaScript, CSS, or videos from a nearby CloudFront edge location instead of repeatedly fetching them from the origin.

Cost-Effective

Does putting servers all over the world make this expensive?

You don't deploy and manage your own servers at all those edge locations. CloudFront provides the global infrastructure for you.

Pricing is pay-as-you-go, meaning there are generally no upfront infrastructure costs just to start using CloudFront. Caching can also reduce the amount of traffic and work reaching your origin.

Without CloudFront	With CloudFront
Many requests reach the origin	Cached requests can be served by CloudFront
Origin handles more traffic	Origin can handle less traffic
You manage scaling at the origin	CloudFront handles global content delivery

What are you actually paying for?

Imagine your website has a 10 MB video stored in S3. A user requests it through CloudFront.

CloudFront may fetch it from the origin, cache it, and then deliver it to users. The traditional usage-based CloudFront pricing model mainly considers things like:

What	What it means
Data transfer to users	How much data CloudFront sends out to viewers
Requests	Number/type of HTTP/HTTPS requests CloudFront handles
Origin-related transfer	Some transfers between CloudFront and origins can have separate pricing rules; AWS-origin transfers often receive favorable/free treatment
Optional features	Some additional CloudFront capabilities can add charges

This means that if 100 people each download a 10 MB file, CloudFront has delivered roughly:

100 x 10 MB = 1 GB

That's the kind of usage that “pay as you go” is talking about.

Caching can also help reduce costs elsewhere. When CloudFront caches that 10 MB file, it does not need to retrieve it from the origin for every user. Instead, many requests can be served from CloudFront's cache.

This reduces traffic and workload on services such as S3, EC2, and Elastic Load Balancing, potentially reducing the amount of infrastructure needed.

NOTE: CloudFront still has its own usage costs. Caching can reduce origin costs, but it does not make content delivery free.

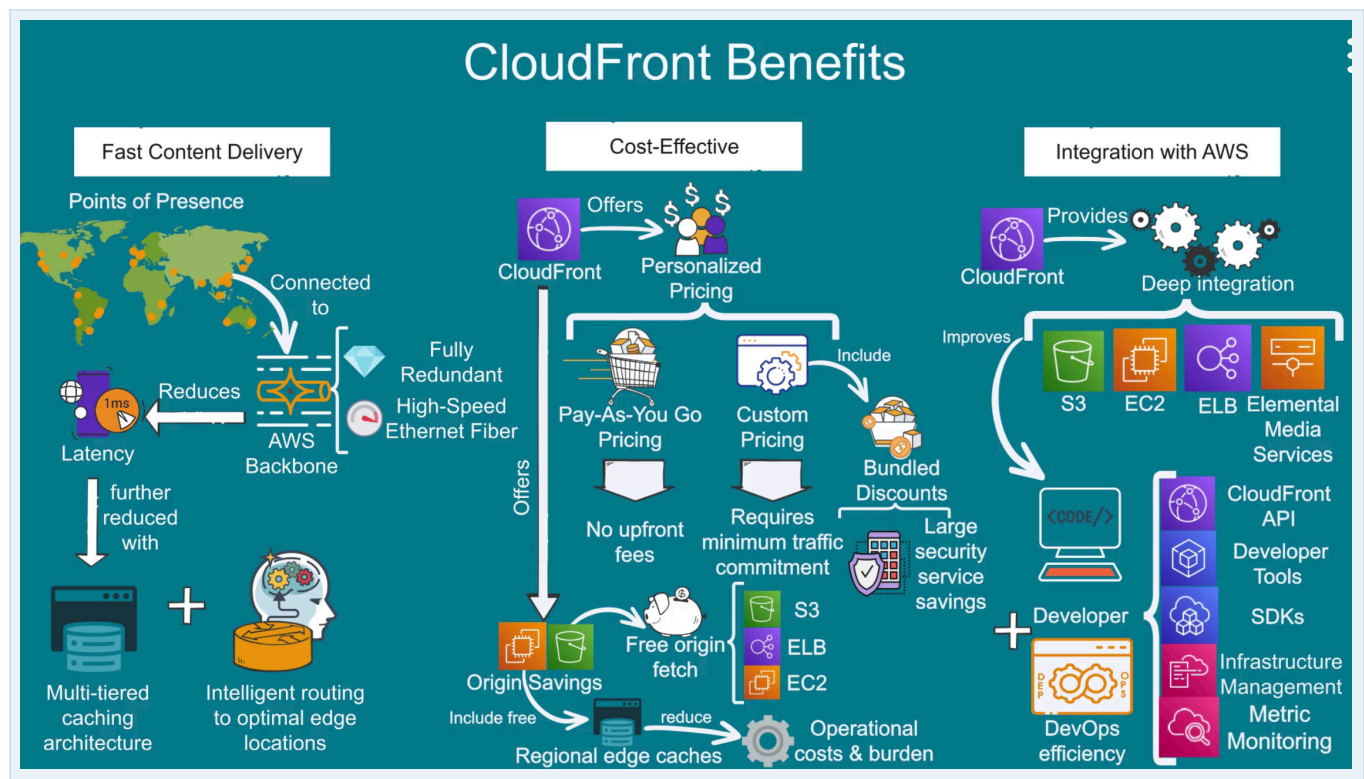
Integration with AWS

How does CloudFront fit with the AWS services we've already learned?

CloudFront integrates with AWS services that can act as or sit behind its origin, including S3 and Elastic Load Balancing.

Service	Responsibility
Route 53	example.com directs users toward the application
CloudFront	Delivers/caches content close to users
ELB	Distributes application requests across healthy servers
EC2	Runs the application
S3	Can store static content such as images/files

CloudFront fits in front of AWS-hosted applications and content to provide global delivery.



Elastic Load Balancing

We can now get users to our application with Route 53 and deliver content faster with CloudFront. But what happens when a request actually reaches our application and we have several servers available?

This is where Elastic Load Balancing (ELB) comes in.

ELB distributes incoming traffic across multiple targets, rather than sending everything to a single resource.

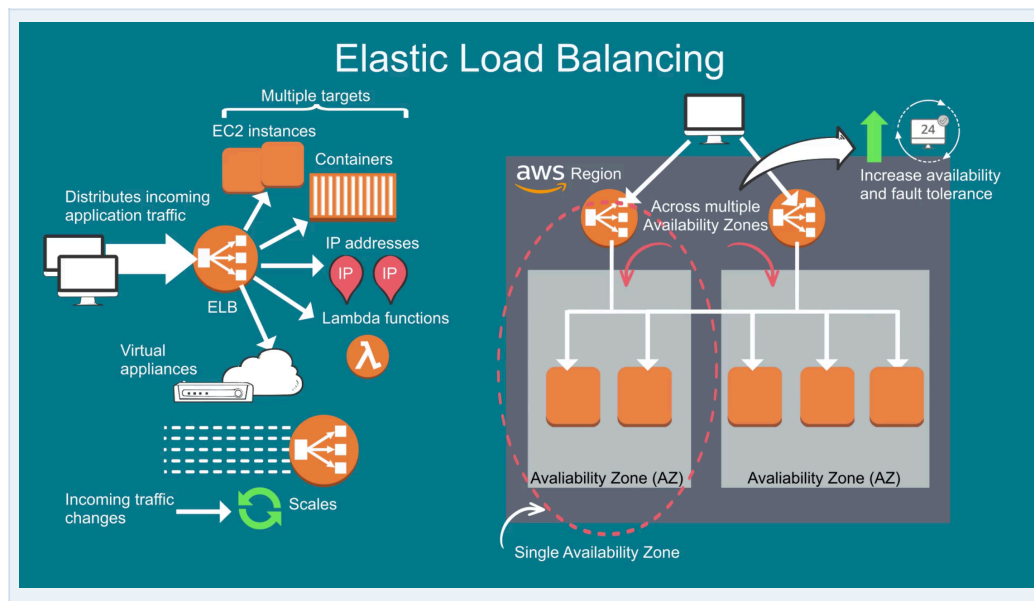
ELB can distribute traffic to	Example use
EC2 instances	Multiple web/application servers
Containers	Containerized applications
IP addresses	Services reachable through specific IPs
Lambda functions	Serverless application functions
Virtual appliances	Network/security appliances

Multiple Availability Zones

A load balancer can distribute traffic across targets in multiple AZs. This improves availability because the application isn't dependent on resources in only one AZ.

If resources in one AZ become unavailable, the load balancer can continue directing traffic to healthy targets in another AZ. This connects directly to our travel-agency problem:

Service	What it solves
Auto Scaling	Makes sure we have enough EC2 instances
ELB	Distributes traffic across those instances
Multiple AZs	Reduces dependence on one physical location
Health checks	Helps ELB avoid unhealthy targets



NOTE: ELB itself also scales automatically as incoming traffic changes. You don't manually increase the size of the load balancer as your application receives more traffic.

Elastic Load Balancing Pricing

So if ELB automatically scales to handle our traffic, what are we actually paying for?

For an Application Load Balancer (ALB), AWS charges for two main things:

You pay for	Meaning
Running time	How long the load balancer is running
Capacity used (LCUs)	How much work/traffic the load balancer processes

An LCU (Load Balancer Capacity Unit) measures load based on things such as connections, processed data, and routing-rule evaluations. AWS bills according to the dimension with the highest usage.

You pay for the load balancer and the EC2 instances behind it; ELB doesn't include the cost of those servers.

How does ELB know where to send traffic?

When traffic reaches a load balancer, AWS needs to know which port to listen on, which servers belong to the application, which ones are healthy, and where each request should go.

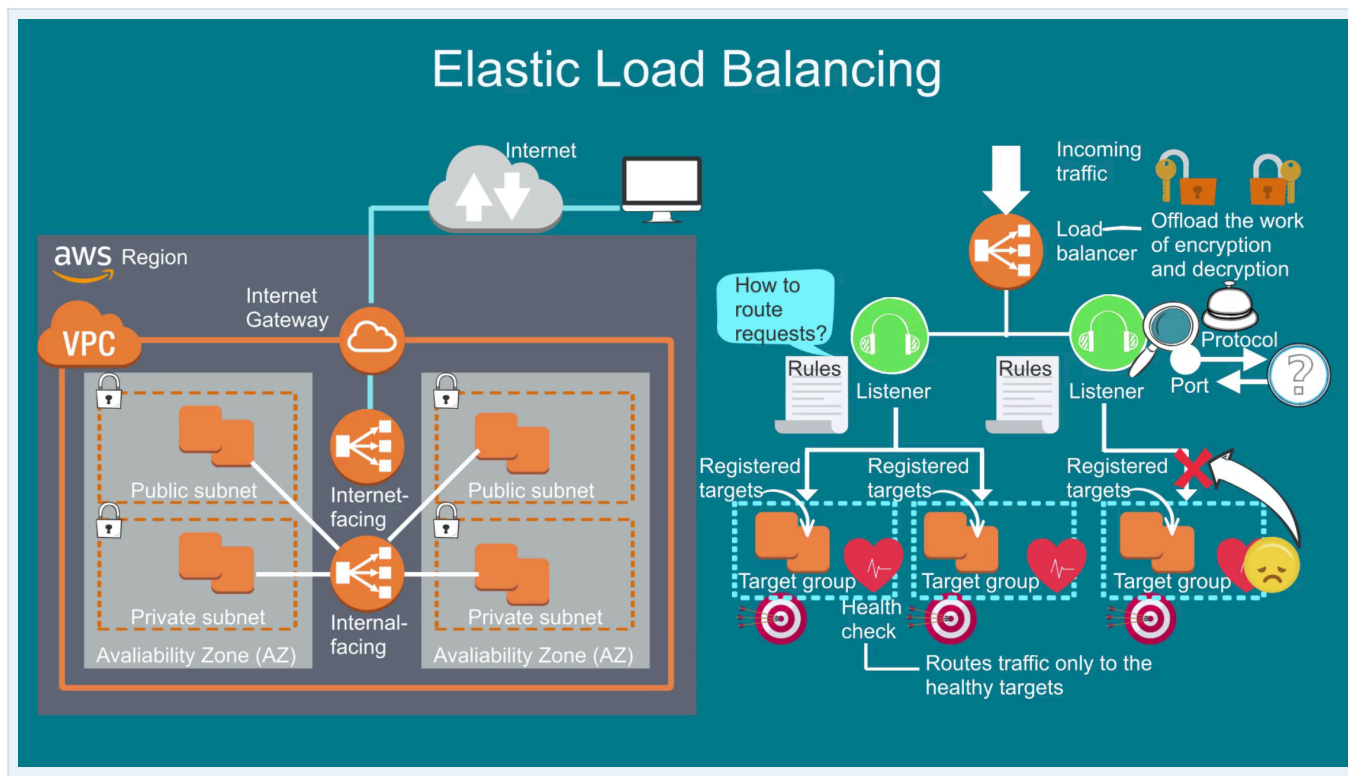
That's where listeners, rules, target groups, and health checks come in.

Component	What it does
Listener	Waits for incoming connections on a specific protocol and port, such as HTTPS on port 443
Rules	Tell the listener how to route matching requests
Target Group	A collection of resources that can receive the traffic, such as EC2 instances
Health Check	Regularly checks whether targets are healthy and able to receive traffic

Example: suppose our travel agency has three EC2 web servers in a target group.

- Listener: HTTPS on port 443
- Rule: Send website traffic to WebServers
- Target group: EC2-1, EC2-2, EC2-3
- Health check: Check that the app responds successfully

If EC2-2 becomes unhealthy, the load balancer stops routing traffic to it and continues using the healthy targets.



NOTE: The left side of the image also introduces another distinction. Internet-facing receives traffic from clients over the internet, while internal distributes traffic between resources inside your private network.

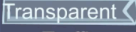
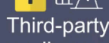
Elastic Load Balancer Types

We know ELB distributes traffic, but why does AWS have several different load balancers?

Because not all traffic needs to be understood in the same way. A normal web application may need intelligent HTTP routing, while another system may care more about extremely fast network connections.

Type	Best suited for	Traffic	Key idea
Application Load Balancer (ALB)	Websites, APIs and modern applications	HTTP, HTTPS, gRPC	Understands application requests and can make advanced routing decisions
Network Load Balancer (NLB)	Very high-performance network traffic	TCP, UDP, TLS	Handles huge numbers of connections with very low latency
Gateway Load Balancer (GWLB)	Network/security appliances	IP	Sends network traffic through appliances such as firewalls
Classic Load Balancer (CLB)	Older AWS applications	TCP, SSL/TLS, HTTP, HTTPS	Previous-generation load balancer; mainly for legacy workloads

Load Balancer Types

						
	Elastic Load Balancing	Application Load Balancer	Network Load Balancer	Gateway Load Balancer	Classic Load Balancer	
Load balancer type	Layer 7 	Layer 4 	Layer 3 Gateway  Layer 4 Load balancing	Layer 4 	Layer 7 	 High availability
Traffic / Protocol	HTTP, HTTPS, gRPC	TCP, UDP, TLS	IP	TCP, SSL, TLS, HTTP, HTTPS		 Automatic scaling
Features	Advanced request routing  Modern application architectures   Based on content of the request	Extreme performance  Millions of requests per second Ultra low latencies  Static IP  One elastic IP per subnet 	 Third-party virtual networking appliances  Auto-scaling   Traffic Source  Destination  Analytics  Third-party appliances 	 EC2-Classical network	 Robust security  Fault tolerant	

Which one would we use?

NOTE: For our travel agency website, the Application Load Balancer (ALB) is the natural choice because it understands HTTP/HTTPS requests. This is Layer 7 routing and can make decisions based on the content of the request. By comparison, an NLB operates at Layer 4 and primarily deals with network connections rather than understanding URL endpoints.

High Availability Diagram

Starting with the Problem

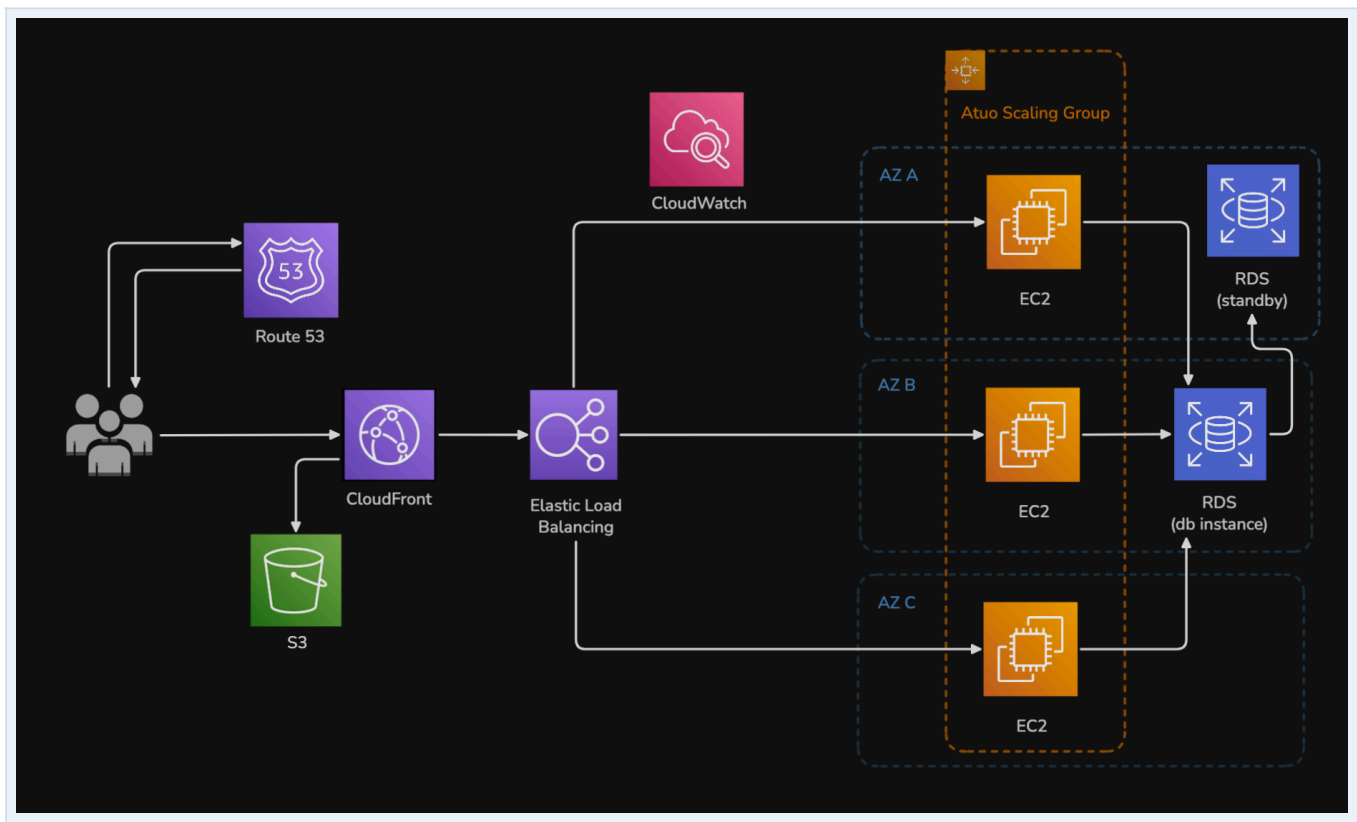
A user wants to visit our travel agency website. What actually happens to their request, and how does AWS keep the application fast and available if traffic increases or something fails?

The travel agency needs a web application that customers can access through a normal domain name such as travelagency.com. But simply putting a website on a server isn't enough.

It needs to be designed so that it:

- Is easy for users to find and access
- Loads quickly for users in different locations
- Can handle sudden increases in traffic
- Continues working if a server or Availability Zone fails
- Keeps the database available if something goes wrong
- Can be monitored so problems are detected

The architecture below combines several AWS services to solve these problems.



Users and Route 53

“How do users find our application?”

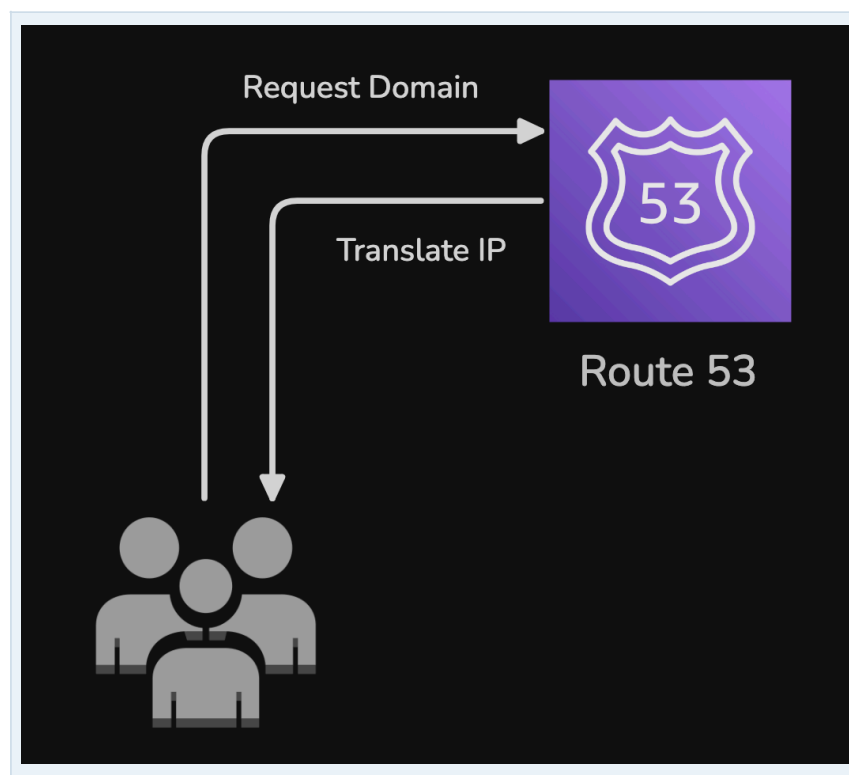
Suppose we've already deployed the travel agency application. There's immediately a problem: how does a customer actually get to it?

Imagine if we hosted it and told our customer to visit `ec2-13-53-123-45.eu-north-1.compute.amazonaws.com`. Not exactly something they are going to remember.

Instead, we want customers to visit something simple like `travelagency.com`.

This creates our first problem: how do we connect a human-friendly domain name to where our application is actually hosted?

This is where Amazon Route 53 comes in. Route 53 provides DNS, allowing us to connect a domain such as the one mentioned above to the destination where our application can be reached.



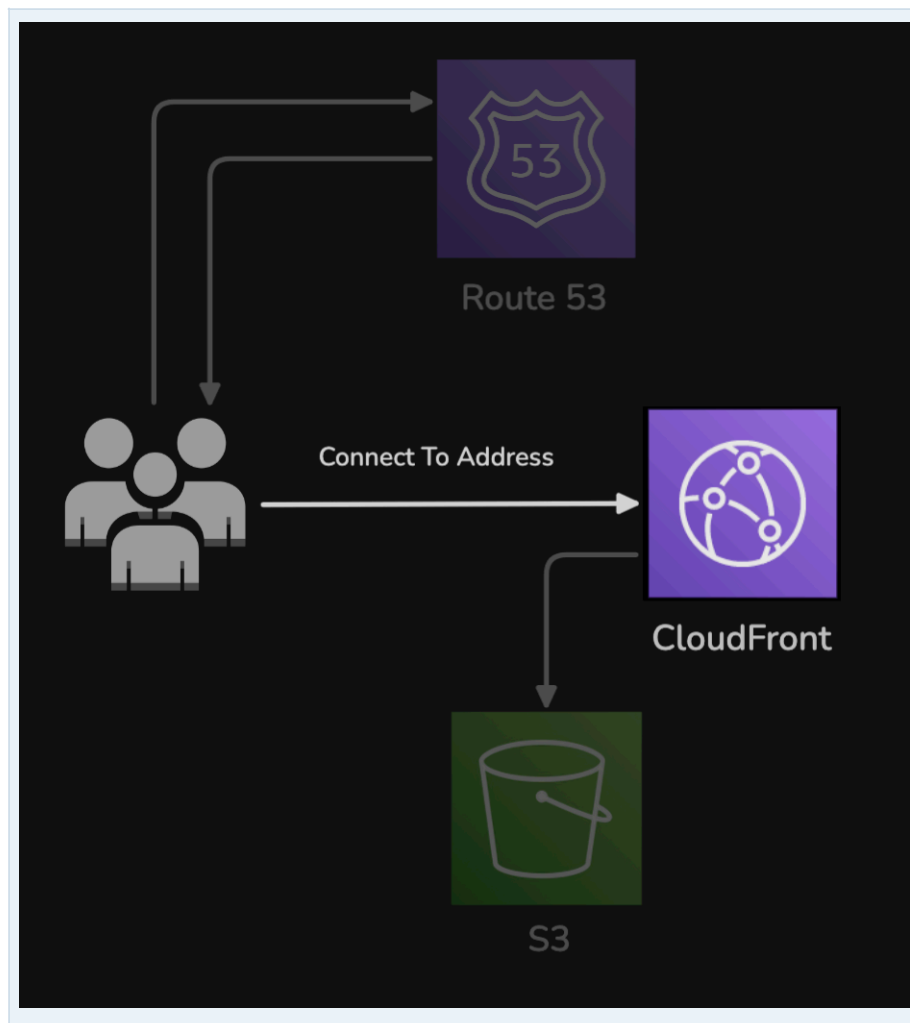
Content Delivery and CloudFront

“How do we make the application fast for users?”

Great, our customers can now enter `travelagency.com`, and Route 53 helps them find the application.

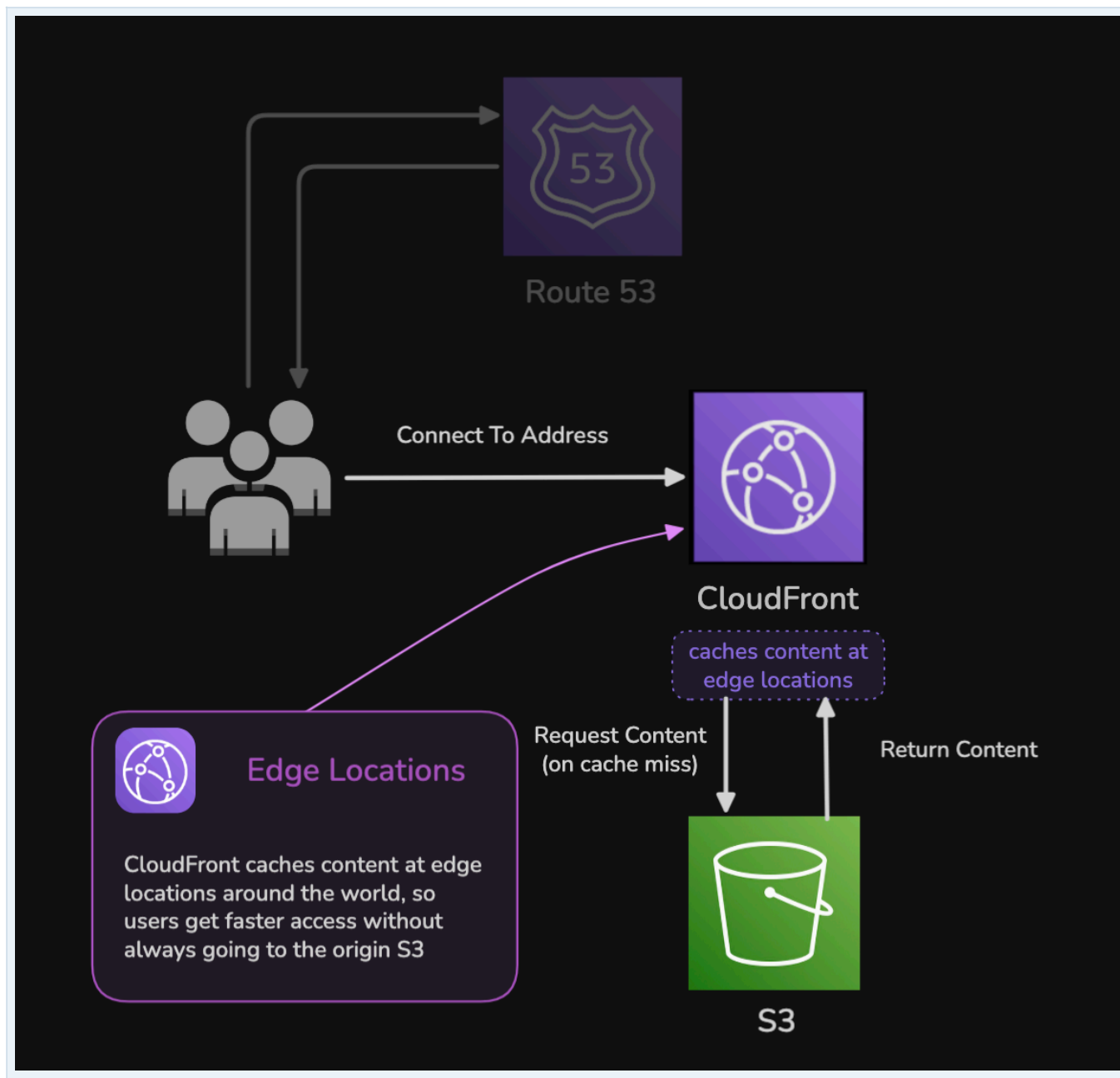
But imagine our application is hosted in Germany and a customer is visiting from Japan. If every image, CSS file, JavaScript file, HTML file, or video has to travel all the way from our origin, the distance can increase latency.

We don't want the location of our application to make the website unnecessarily slow for users farther away. This is where Amazon CloudFront comes in.



CloudFront is AWS's Content Delivery Network (CDN). It uses a global network of edge locations to cache and deliver content closer to users.

For example, the travel agency could store destination images in Amazon S3. The first time an image is requested, CloudFront can retrieve it from S3 and cache a copy closer to the users requesting it. Later requests can then be served from CloudFront's cache instead of repeatedly going back to S3.



Elastic Load Balancing

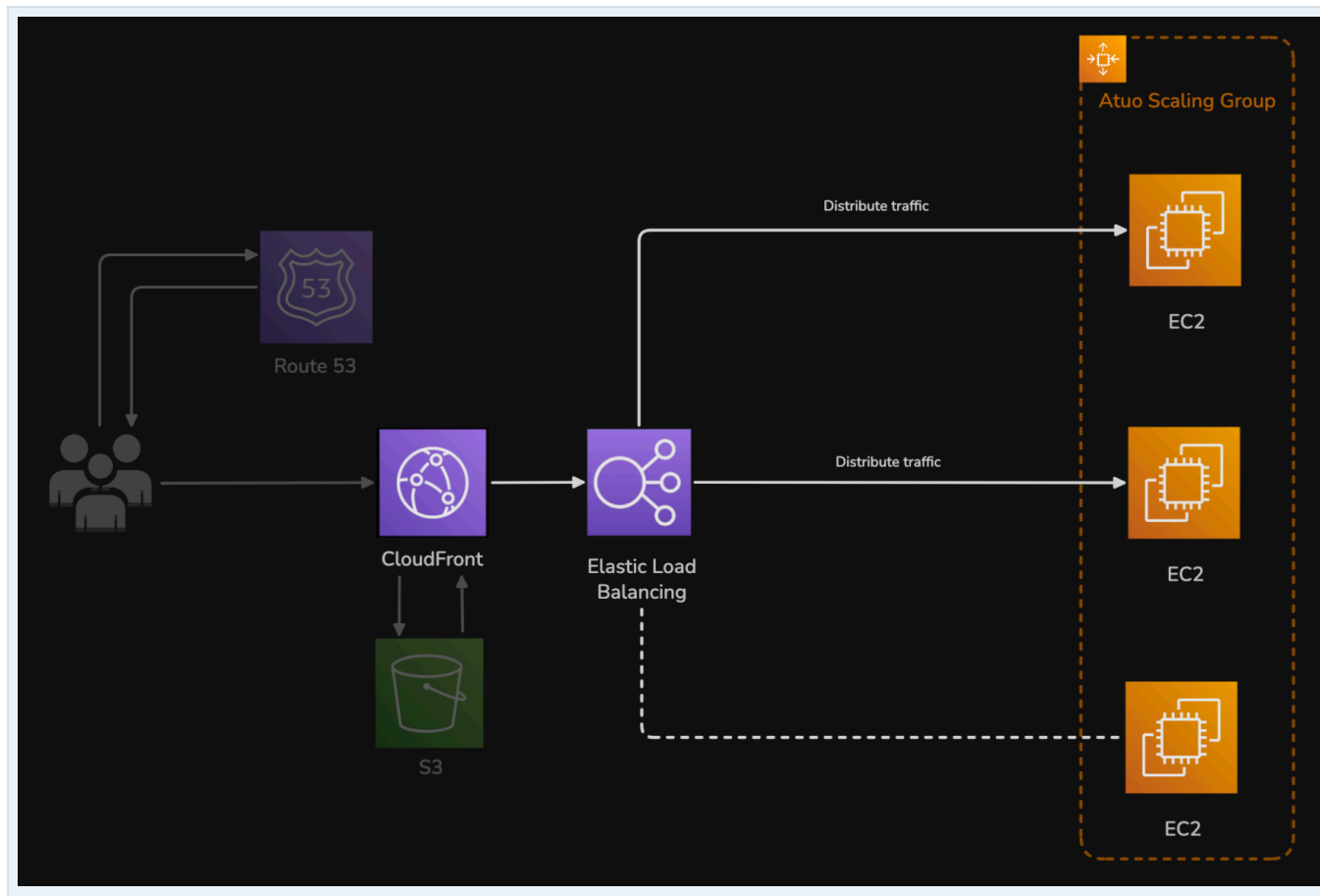
“How do we distribute traffic between our servers?”

CloudFront helps us deliver content quickly, but our application still needs servers to actually process requests.

If we only had one EC2 instance, every application request would be sent to that one server. This can be fine for portfolios and smaller web apps that expect little traffic. If traffic suddenly increased, however, that server could become overloaded. And if it failed, the application could become unavailable.

Instead, running multiple EC2 instances across multiple Availability Zones makes the web application more available. But this creates another problem: which EC2 instance should receive each request?

This is where Elastic Load Balancing (ELB) comes in.



NOTE: ELB sits in front of EC2 instances and distributes incoming application traffic across healthy instances. If one instance becomes unhealthy, the load balancer can stop sending traffic to it and continue using healthy instances.

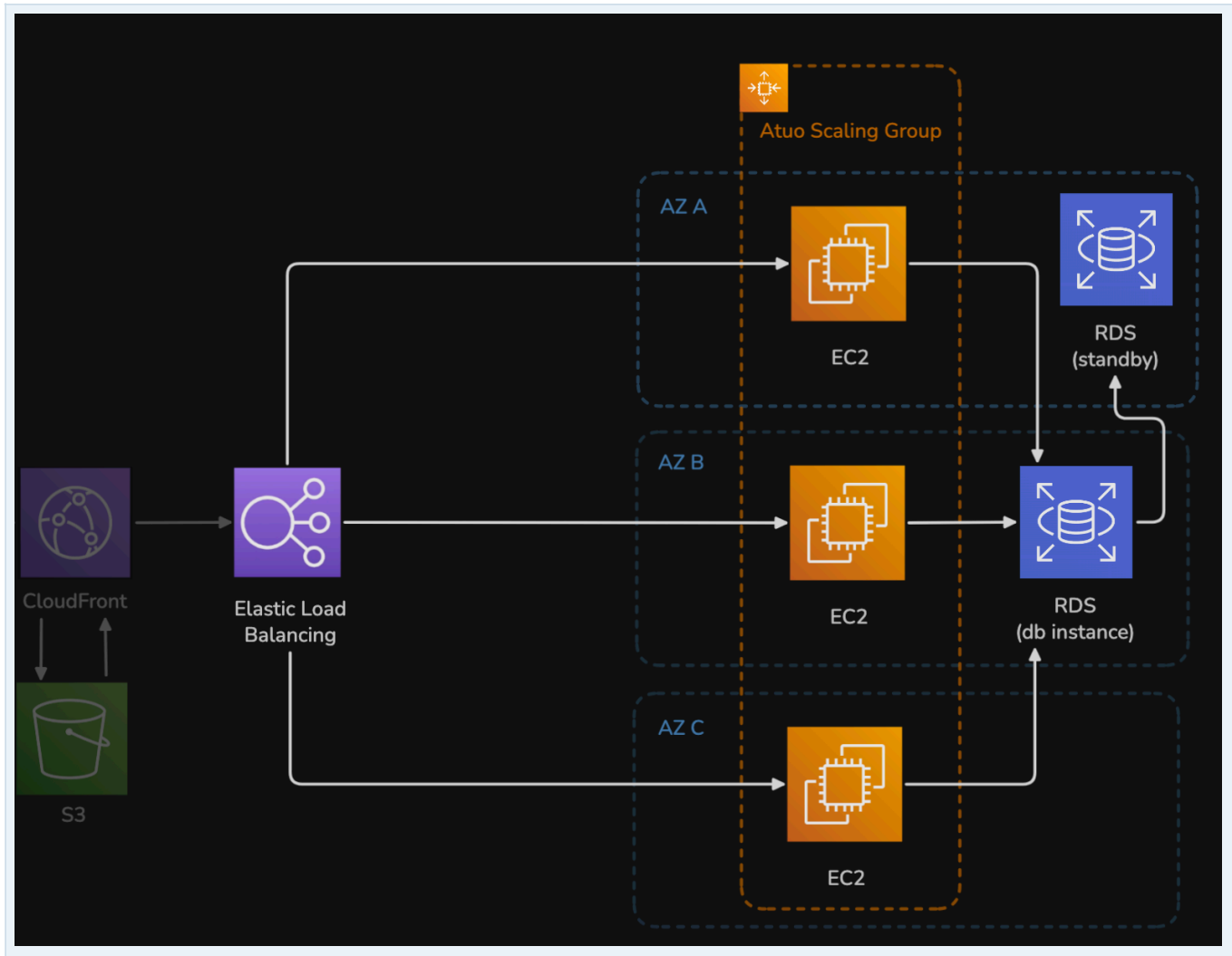
NOTE: The ELB itself can perform health checks.

Availability Zones

“What happens if part of our infrastructure goes down?”

Our load balancer can distribute traffic between multiple EC2 instances, but there is still a problem.

Imagine all three EC2 instances were running in the same Availability Zone. If that Availability Zone experienced an outage, all three servers could become unavailable at the same time.

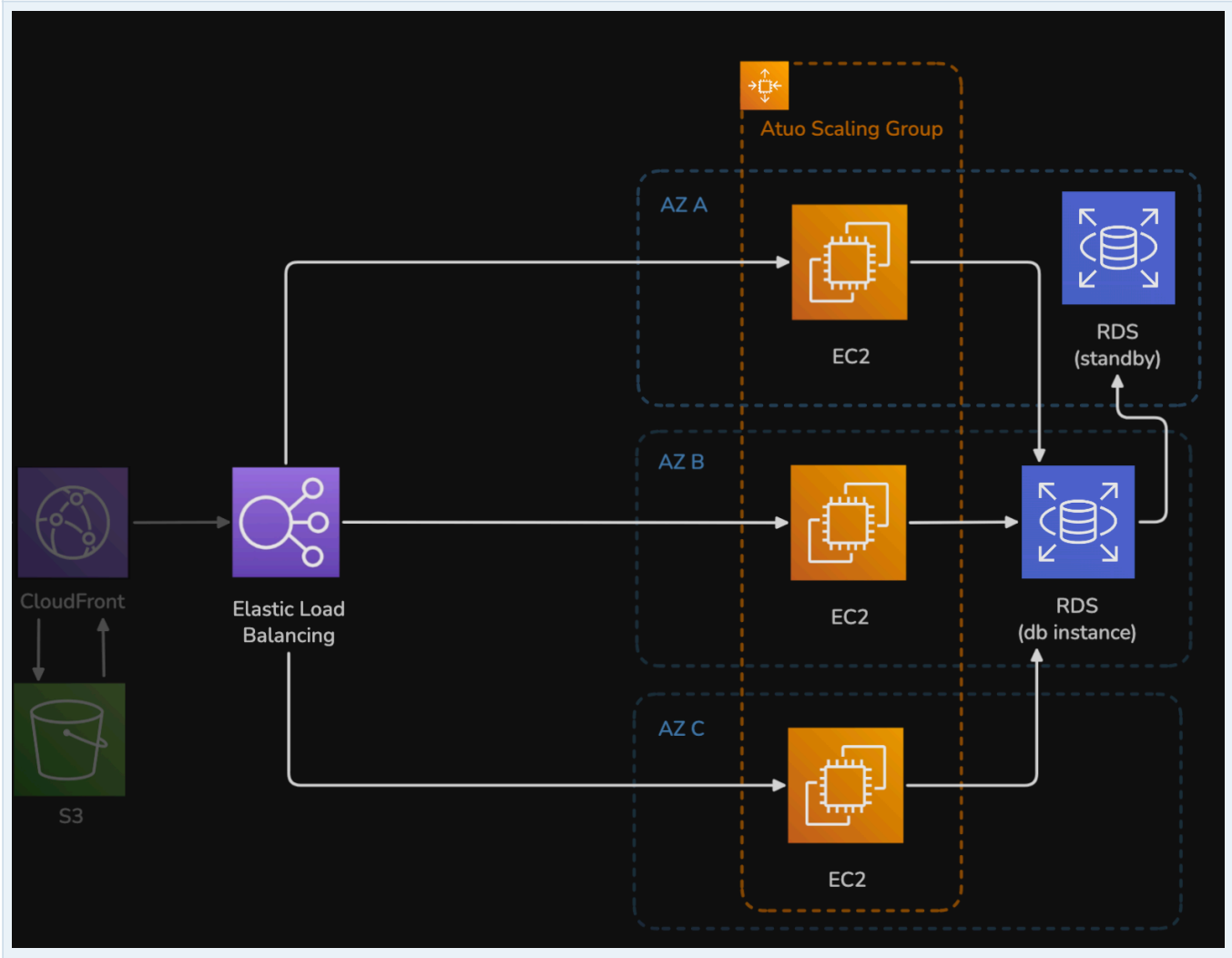


NOTE: In this architecture, EC2 instances are running across AZ A, B, and C. If one AZ becomes unavailable, the load balancer can continue sending traffic to healthy EC2 instances in the other AZs.

RDS and Multi-AZ

“But what happens if our database fails?”

Our EC2 instances are now spread across multiple Availability Zones, but they all depend on the application's database.



If we only had one RDS database instance and it became unavailable, our EC2 servers might still be running, but they could no longer access the data they need. To reduce this risk, Amazon RDS can use a Multi-AZ deployment.

Like in the earlier modules, RDS maintains a primary database instance in one AZ and a standby database in another. Changes made to the primary are synchronously replicated to the standby.

If the primary database becomes unavailable, RDS can fail over to the standby.

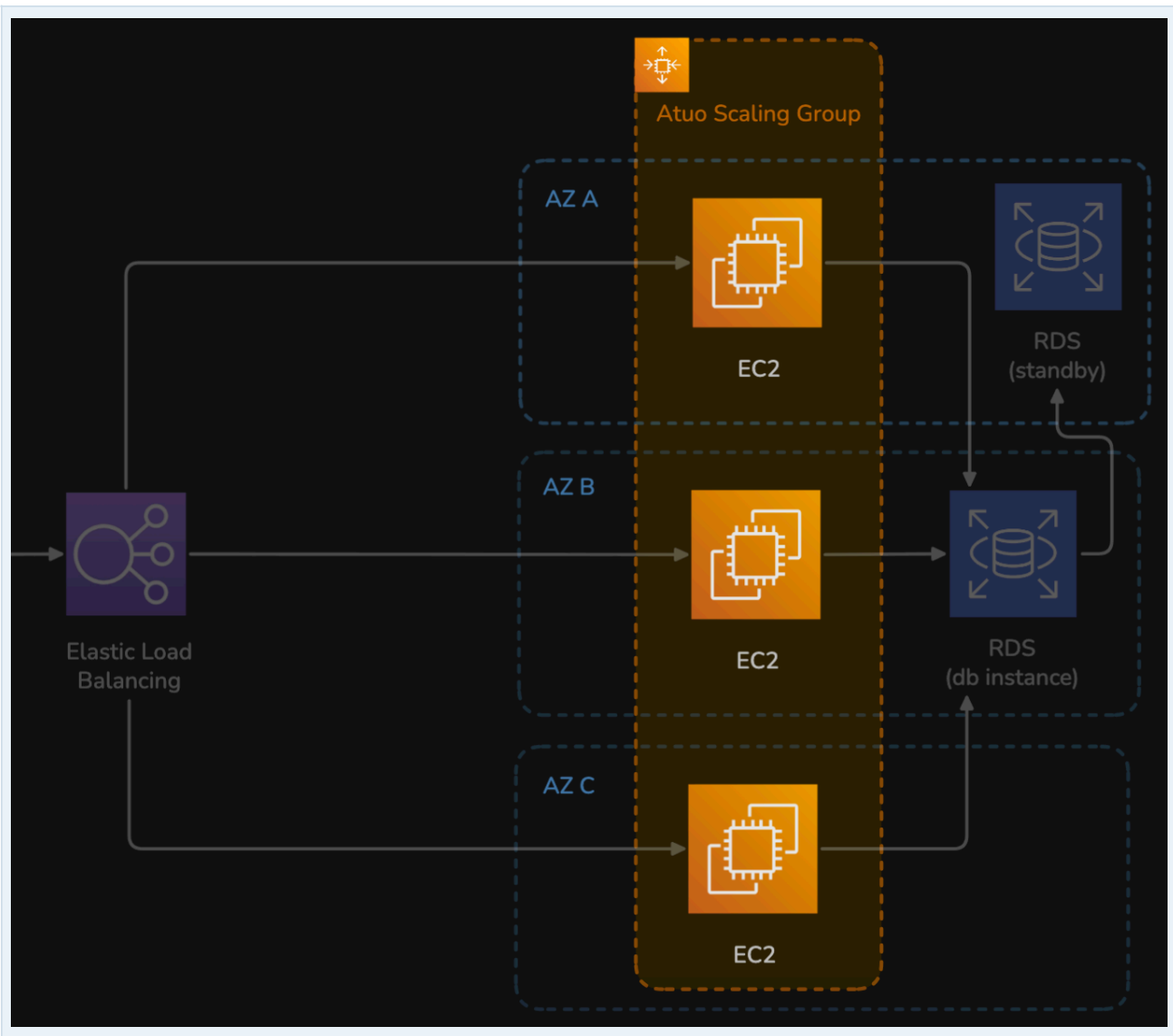
Auto Scaling

“What happens when we need more or fewer EC2 instances?”

We have multiple Availability Zones, and the load balancer can distribute traffic between our healthy EC2 instances. But the amount of traffic the travel agency receives won't always be the same.

Imagine they normally need three EC2 instances, but then launch a major summer promotion and thousands of additional customers visit the website. The existing instances might not have enough capacity to handle the increased traffic.

An Auto Scaling group can automatically manage a group of EC2 instances and adjust the number of instances according to scaling policies.



NOTE: An Auto Scaling Group handles EC2 instances, but it uses a Launch Template to create new EC2 instances. The Launch Template specifies the AMI (image) to use and can define the EBS volumes attached to each instance.

CloudWatch

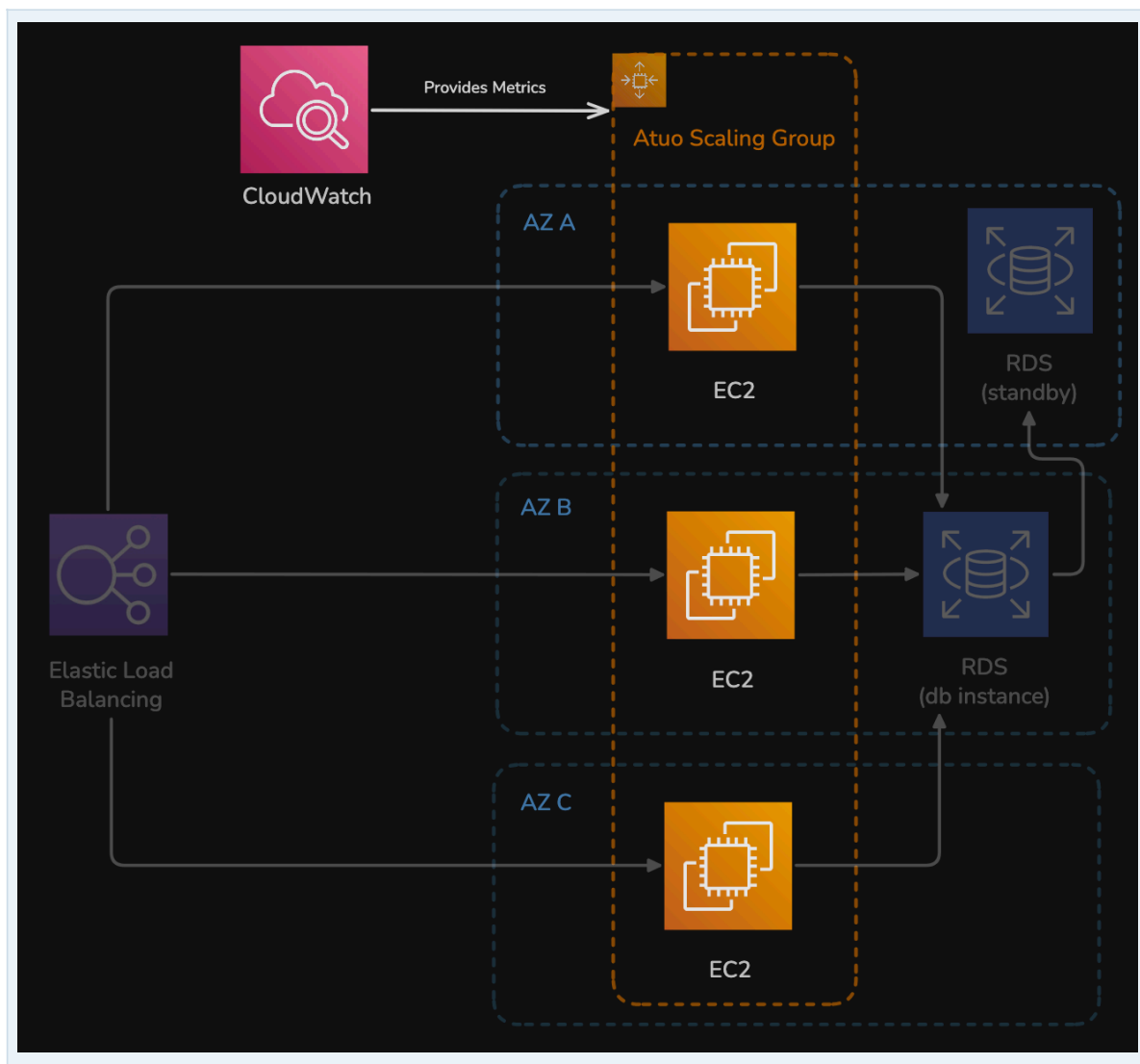
“How do we know when our application needs to scale?”

While the Auto Scaling group can add or remove EC2 instances, it still needs information about how our instances are performing.

Imagine traffic increases and the average CPU utilisation of the EC2 instances reaches 80%. Amazon CloudWatch collects and monitors metrics from AWS resources, such as EC2 CPU utilisation.

These metrics can be used by our Auto Scaling policies. If a configured threshold or target is reached, Auto Scaling can respond by adding or removing EC2 instances.

CloudWatch can also create alarms, allowing us to detect problems and trigger actions when certain conditions are met.



NOTE: CloudWatch monitors demand, Auto Scaling adjusts EC2 capacity, and ELB distributes traffic across the healthy EC2 instances.

02 Practice Lab

In this practice lab we will:

- Configure an Auto Scaling group to use an Application Load Balancer.
- Configure load balancer health checks for the Auto Scaling group.
- Add a second Availability Zone to the Auto Scaling group.

Review the Existing Auto Scaling Group

Begin by checking EC2 instances > Auto Scaling Groups and reviewing the contents. In our case, we already have a predefined Auto Scaling group.

The screenshot shows the AWS Auto Scaling Groups console. At the top, it says 'Auto Scaling groups (1/1) Info' with a refresh button and a timestamp 'Last updated less than a minute ago'. There are buttons for 'Launch configurations', 'Launch templates', 'Actions', and 'Create Auto Scaling group'. Below is a search bar 'Search your Auto Scaling groups'. A table lists the groups:

☒	Name	Status - new	Launch template/configuration	Instances	Instance health - new
☒	TravelAgencyWebServers	At desired capacity	TravelAgencyTemplate-80f21810 Versio 1		1/1 Healthy

When reviewing the network settings, we can see that we only have one Availability Zone: us-east-1a.

The screenshot shows the 'Network' settings for the 'TravelAgencyWebServers' group. It includes an 'Edit' button. The settings are:

Availability Zones	Subnet ID	Availability Zone distribution
use1-az6 (us-east-1a)	subnet-Od30a12c984e8acf8	Balanced best effort

This is bad for the high-availability architecture we've just learned. Even if Auto Scaling launches 10 EC2 instances, they could all be in AZ A. If AZ A goes down, we potentially lose all 10.

When navigating to the Instances tab, we can see that the desired capacity appears to be 1 EC2 instance.

The screenshot shows the 'Auto Scaling group: TravelAgencyWebServers' console with the 'Instance management' tab selected. It has tabs for 'Details', 'Integrations', 'Automatic scaling', 'Instance management', 'Instance refresh', 'Activity', 'Monitoring', and 'Tags'. Below is a section for 'Instances (1)' with a refresh button and an 'Actions' dropdown. A search bar 'Filter instances' is present. A table lists the instances:

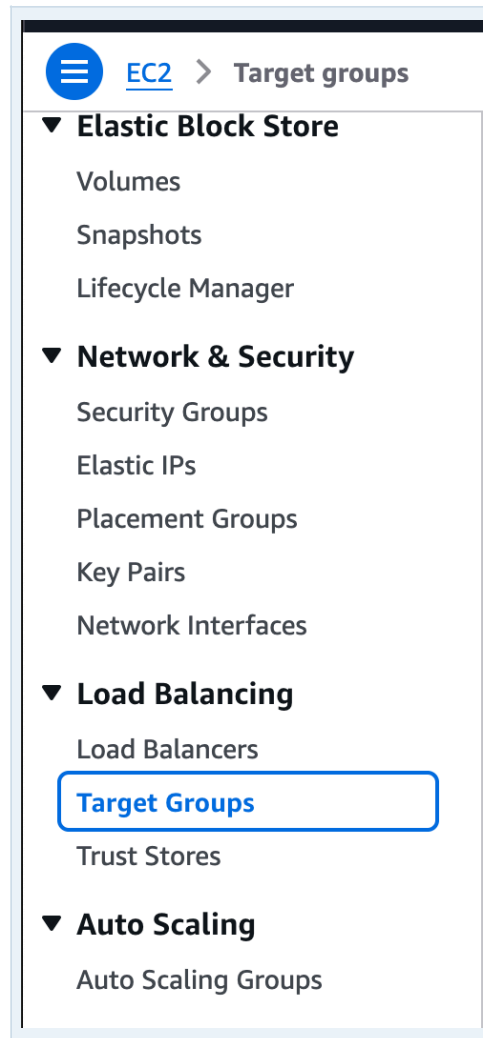
☐	Instance ID	Lifecycle	Instanc...	Weight...	Launch...	Availab...	Health ...	Protect...
☐	i-072b10b1897cc32f6	InService	t3.micro	-	TravelAgencyTe	use1-az6 (...)	Healthy	

Create a Target Group

In order to connect the load balancer with the Auto Scaling architecture, a Target Group is required. It is essentially the ELB's pool of destinations.

So currently we have an Auto Scaling Group with EC2 instances. But the load balancer still needs to know:

“Which EC2 instances am I allowed to send traffic to?”



Navigate and click: Create target group.

Target Type	Load Balancer sends traffic to...	Easy Example
Instances	EC2 instances	Your travel agency EC2 instances
IP addresses	Specific IP addresses	10.10.1.15
Lambda function	A Lambda function	Serverless application
Application Load Balancer	Another ALB	NLB > ALB > application

Settings - immutable

Choose a target type and the load balancer and listener will route traffic to your target. These settings can't be modified after target group creation.

Target type
Indicate what resource type you want to target. Only the selected resource type can be registered to this target group.

☒ **Instances**

Supports load balancing to instances in a VPC. Integrate with Auto Scaling Groups or ECS services for automatic management.

Suitable for: ALB NLB GWLB

☐ **IP addresses**

Supports load balancing to VPC and on-premises resources. Facilitates routing to IP addresses and network interfaces on the same instance. Supports IPv6 targets.

Suitable for: ALB NLB GWLB

☐ **Lambda function**

Supports load balancing to a single Lambda function. ALB required as traffic source.

Suitable for: ALB

☐ **Application Load Balancer**

Allows use of static IP addresses and PrivateLink with an Application Load Balancer. NLB required as traffic source.

Suitable for: NLB

NOTE: Keep Instances selected, as we'll be working with EC2 instances.

AWS Cloud • Part X

Keep IPv4 and HTTP, and name the target group TravelAgencyWebServer-TG.

Target group name
Name must be unique per Region per AWS account.

Accepts: a-z, A-Z, 0-9, and hyphen (-). Can't begin or end with hyphen. 1-32 total characters; Count: 24/32

Protocol
Protocol for communication between the load balancer and targets.

Port
Port number where targets receive traffic. Can be overridden for individual targets during registration.

1-65535

IP address type
Only targets with the indicated IP address type can be registered to this target group.

☒ **IPv4**
Each instance has a default network interface (eth0) that is assigned the primary private IPv4 address. The instance's primary private IPv4 address is the one that will be applied to the target.

☐ **IPv6**
Each instance you register must have an assigned primary IPv6 address. This is configured on the instance's default network interface (eth0). [Learn more](#)

Make sure to pick the relevant VPC.

VPC
Select the VPC with the instances that you want to include in the target group. Only VPCs that support the IP address type selected above are available in this list.

10.0.0.0/16
[Create VPC](#)

Protocol version

☒ **HTTP1**
Send requests to targets using HTTP/1.1. Supported when the request protocol is HTTP/1.1 or HTTP/2.

☐ **HTTP2**
Send requests to targets using HTTP/2. Supported when the request protocol is HTTP/2 or gRPC, but gRPC-specific features are not available.

☐ **gRPC**
Send requests to targets using gRPC. Supported when the request protocol is gRPC.

Health Checks

Scrolling down to Health checks, we can see that the Load Balancer periodically checks targets in the Target Group to decide whether they should receive traffic.

Health checks

The associated load balancer periodically sends requests, per the settings below, to the registered targets to test their status.

Health check protocol

HTTP ▼

Health check path

Use the default path of "/" to perform health checks on the root, or specify a custom path if preferred.

/

Up to 1024 characters allowed.

► **Advanced health check settings**

Since we're using the defaults, we're using HTTP and path '/'. Conceptually, the load balancer repeatedly does:

GET / (EC2 instance)

If the instance responds successfully enough times, it will be considered healthy. Leave the defaults unchanged and move forward.

Scrolling further down reveals Target Optimizer, which is a more advanced optional feature. Normally, the load balancer distributes requests among healthy targets. But imagine one EC2 instance is already processing a large number of expensive requests at the same time. Target Optimizer lets you impose a maximum number of concurrent requests per target.

Target optimizer - optional [Info](#)

Enforce a maximum number of concurrent requests on a target. This is particularly useful for compute-intensive workloads like complex data processing or inference.

Target optimizer

☒ **Off - Default**
Use standard load balancing.

☐ **On**
Enforce a maximum number of concurrent requests on a target. Requires targets to have target optimizer agent installed.

Attributes

ⓘ Certain default attributes will be applied to your target group. You can view and edit them after creating the target group.

► **Tags - optional**

Consider adding tags to your target group. Tags enable you to categorize your AWS resources so you can more easily manage them.

[Cancel](#) [Next](#)

For example:

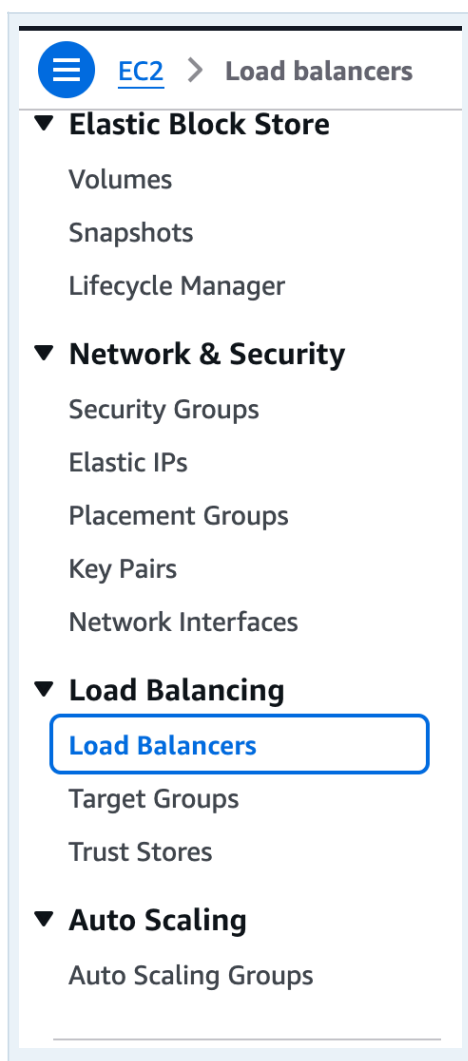
- EC2 A > already handling the maximum concurrent requests.
- EC2 B > still has capacity.

Do not send another request to EC2 A yet.

In our case, we won't change anything. Continue through the wizard and create the target group. Verify that it was created successfully.

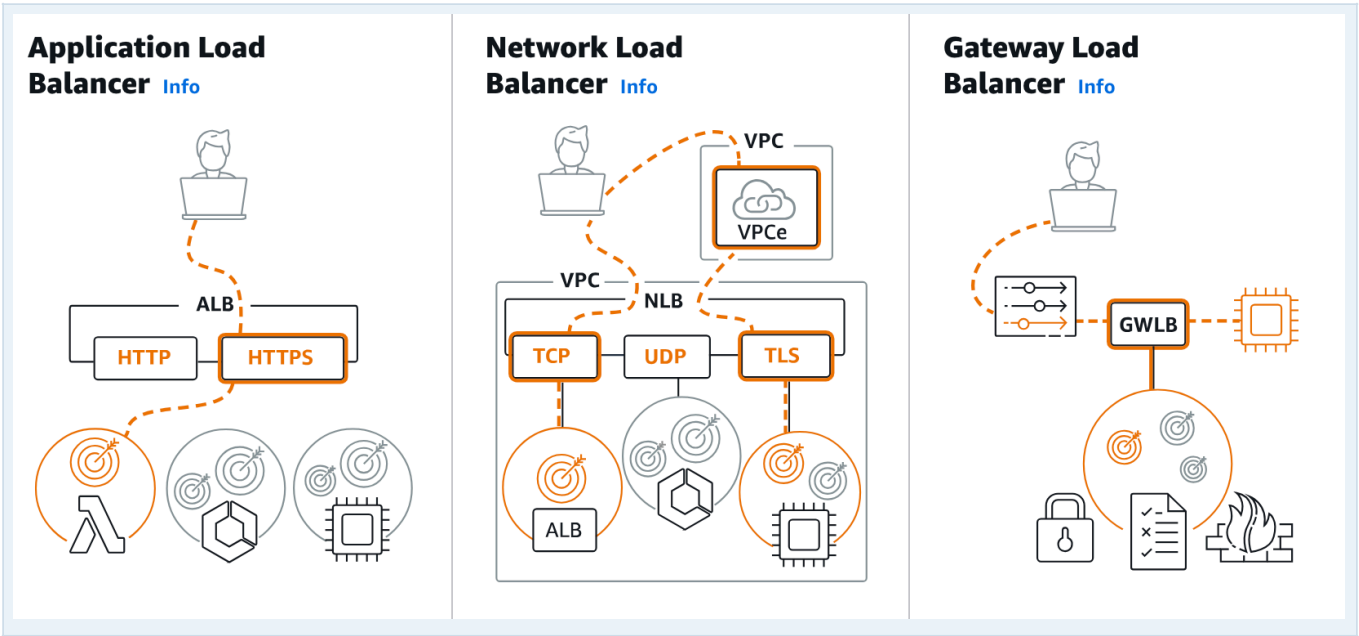
Create an Application Load Balancer

Next, navigate to Load Balancers.



Click Create load balancer.

Type	Understands	Best for	Simple example
Application Load Balancer (ALB)	HTTP, HTTPS, gRPC	Websites, APIs, web applications	/hotels > hotel EC2 instances
Network Load Balancer (NLB)	TCP, UDP, TLS	Very high-performance network traffic	Game server connections
Gateway Load Balancer (GWLB)	IP traffic	Security/network appliances	Send traffic through firewalls



Since we're working with a web application and application-level web requests, an Application Load Balancer makes sense. It operates at Layer 7 of the OSI model.

Name it TravelAgencyServers-LB and make it internet-facing.

Basic configuration

Load balancer name
Name must be unique within your AWS account and can't be changed after the load balancer is created.

A maximum of 32 alphanumeric characters including hyphens are allowed, but the name must not begin or end with a hyphen.

Scheme | [Info](#)
Scheme can't be changed after the load balancer is created.

☒ **Internet-facing**

- Serves internet-facing traffic.
- Has public IP addresses.
- DNS name resolves to public IPs.
- Requires a public subnet.

☐ **Internal**

- Serves internal traffic.
- Has private IP addresses.
- DNS name resolves to private IPs.
- Compatible with the IPv4 and Dualstack IP address types.

Load balancer IP address type | [Info](#)
Select the front-end IP address type to assign to the load balancer. The VPC and subnets mapped to this load balancer must include the selected IP address types. Public IPv4 addresses have an additional cost.

☒ **IPv4**
Includes only IPv4 addresses.

☐ **Dualstack**
Includes IPv4 and IPv6 addresses.

☐ **Dualstack without public IPv4**
Includes a public IPv6 address, and private IPv4 and IPv6 addresses. Compatible with internet-facing load balancers only.

The Load Balancer should be in the same VPC as the EC2 instances it is sending traffic to.

Network mapping

 | [Info](#)

The load balancer routes traffic to targets in the selected subnets, and in accordance with your IP address settings.

VPC | [Info](#)
The load balancer will exist and scale within the selected VPC. The selected VPC is also where the load balancer targets must be hosted unless routing to Lambda or on-premises targets, or if using VPC peering. To confirm the VPC for your targets, view [target groups](#).

vpc-0176392ce06dbb73d (lab/TravelAgencyVpc)
10.0.0.0/16

▼

 [Create VPC](#)

Select all relevant Availability Zones and make sure you choose public subnets.

☒ **us-east-1a (use1-az6)**

Subnet
Only CIDR blocks corresponding to the load balancer IP address type are used. At least 8 available IP addresses are required for your load balancer to scale efficiently. Selected subnets must have Network ACL inbound and outbound ALLOW rules, and internet gateway (IGW) routes configured to allow traffic.

subnet-0d30a12c984e8acf8 (lab/TravelAgencyVpc/PublicSubnet1)
10.0.0.0/19
Network ACL ALLOW rules: Inbound (1) Outbound (1) | Route table: IGW (✓)

▼

☒ **us-east-1b (use1-az1)**

Subnet
Only CIDR blocks corresponding to the load balancer IP address type are used. At least 8 available IP addresses are required for your load balancer to scale efficiently. Selected subnets must have Network ACL inbound and outbound ALLOW rules, and internet gateway (IGW) routes configured to allow traffic.

subnet-0e326e0401beef925 (lab/TravelAgencyVpc/PublicSubnet2)
10.0.32.0/19
Network ACL ALLOW rules: Inbound (1) Outbound (1) | Route table: IGW (✓)

▼

☒ **us-east-1c (use1-az2)**

Subnet
Only CIDR blocks corresponding to the load balancer IP address type are used. At least 8 available IP addresses are required for your load balancer to scale efficiently. Selected subnets must have Network ACL inbound and outbound ALLOW rules, and internet gateway (IGW) routes configured to allow traffic.

subnet-04cd4f0bf468c1b46 (lab/TravelAgencyVpc/PublicSubnet3)
10.0.64.0/19
Network ACL ALLOW rules: Inbound (1) Outbound (1) | Route table: IGW (✓)

▼

NOTE: Users are coming from the public internet, so the ALB needs a path to and from the internet. Pick public subnets.

Security Groups

Remove the default Security Group and replace it with the relevant Security Group.

Security groups [Info](#)
A security group is a set of firewall rules that control the traffic to your load balancer. Listed security groups are in the same VPC as you selected above. Selected security groups must include at least 1 inbound rule and 1 outbound rule to allow traffic to your load balancer.

Security groups
Select up to 5 security groups

TravelAgencyWebServer (sg-05b4d49f3d680129d) [X](#)
Inbound rule (1) Outbound rule (2)

[Create security group](#)

NOTE: We'll replace this later with the relevant Load Balancer Security Group.

Listeners and Routing

Think of a Listener as the ALB's front door: “What incoming traffic should I listen for?”

Listeners and routing [Info](#)
A listener is a process that checks for connection requests using the port and protocol you configure. The rules that you define for a listener determine how the load balancer routes requests to its registered targets.

▼ Listener HTTP:80 [Remove](#)

Protocol HTTP

Port 80
1-65535

Default action [Info](#)
The default action is used if no other rules apply. Choose the default action for traffic on this listener.

Routing action
☒ Forward to target groups ☐ Redirect to URL ☐ Return fixed response

Forward to target group [Info](#)
Choose a target group and specify routing weight or [create target group](#).

Target group TravelAgencyWebServer-TG
Target type: Instance, IPv4 | Target stickiness: Off

HTTP

[C](#)

Weight 1
0-999

Percent 100%

Here we pick HTTP: 80, so the ALB listens for incoming HTTP requests on port 80.

After receiving the request, it needs to know what to do with it. In this case, we forward it to the Target Group TravelAgencyWebServer-TG.

Settings - immutable

Choose a target type and the load balancer and listener will route traffic to your target. These settings can't be modified after target group creation.

Target type

Indicate what resource type you want to target. Only the selected resource type can be registered to this target group.

☒ Instances

Supports load balancing to instances in a VPC. Integrate with Auto Scaling Groups or ECS services for automatic management.

Suitable for: ALB NLB GWLB

☐ IP addresses

Supports load balancing to VPC and on-premises resources. Facilitates routing to IP addresses and network interfaces on the same instance. Supports IPv6 targets.

Suitable for: ALB NLB GWLB

☐ Lambda function

Supports load balancing to a single Lambda function. ALB required as traffic source.

Suitable for: ALB

☐ Application Load Balancer

Allows use of static IP addresses and PrivateLink with an Application Load Balancer. NLB required as traffic source.

Suitable for: NLB

Remember that our target type was Instances. TravelAgencyWebServer-TG is essentially the pool of EC2 instances that may receive traffic.

Once that's done, scroll to the bottom of the page and review the summary.

Summary

Review and confirm your configurations. [Estimate cost](#)

Basic configuration [Edit](#)

Name: TravelAgencyServers-LB
Scheme: Internet-facing
IP address type: IPv4

Network mapping [Edit](#)

VPC: [vpc-0176392ce06dbb73d](#)
Public IPv4 IPAM pool: -
Availability Zones and subnets:

- us-east-1a
[subnet-0d30a12c984e8acf8](#)
lab/TravelAgencyVpc/PublicSubnet1
- us-east-1b
[subnet-0e326e0401beef925](#)
lab/TravelAgencyVpc/PublicSubnet2
- us-east-1c
[subnet-04cd4f0bf468c1b46](#)
lab/TravelAgencyVpc/PublicSubnet3

Security groups [Edit](#)

TravelAgencyWebServer
[sg-05b4d49f3d680129d](#)

Listeners and routing [Edit](#)

HTTP:80 | Forward to 1 target group

Service integrations [Edit](#)

Amazon CloudFront + AWS Web Application Firewall (WAF): -
AWS WAF: -
AWS Global Accelerator: -

Tags [Edit](#)

-

Then click Create.

✔ Successfully created load balancer: **TravelAgencyServers-LB**
It might take a few minutes for your load balancer to fully set up and route traffic. Targets will also take a few minutes to complete the registration process and pass initial health checks.

ⓘ Introducing new Capacity Utilization metric for ALB
Monitor how much of your load balancer's available capacity was used to meet traffic demand. Visit the **Capacity** or **Monitoring** tabs to view utilization data.

TravelAgencyServers-LB

⌂ Actions ▾

▼ Details

Load balancer type Application	Status 🔄 Provisioning	VPC vpc-0176392ce06dbb73d 🔗	Load balancer IP address type IPv4
Scheme Internet-facing	Hosted zone Z35SXDOTRQ7X7K	Availability Zones subnet-0e326e0401beef925 🔗 us-east-1b (use1-az1) subnet-0d30a12c984e8acf8 🔗 us-east-1a (use1-az6) subnet-04cd4f0bf468c1b46 🔗 us-east-1c (use1-az2)	Date created September 11, 2026, 14:36 (UTC+02:00)

Connect the Load Balancer to Auto Scaling

Navigate to Auto Scaling Groups.

☰ EC2 > Auto Scaling groups

▼ Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

▼ Network & Security

Security Groups

Elastic IPs

Placement Groups

Key Pairs

Network Interfaces

▼ Load Balancing

Load Balancers

Target Groups

Trust Stores

▼ Auto Scaling

Auto Scaling Groups

Select the group.

Auto Scaling groups (1/1) [Info](#)

Last updated less than a minute ago

[Launch configurations](#) [Launch templates](#) [Actions](#) [Create Auto Scaling group](#)

< 1 >

☒	Name	Status - new	Launch template/configuration	Instances	Instance health - new
☒	TravelAgencyWebServers	At desired capacity	TravelAgencyTemplate-7f44f7c0 Version 1		1/1 Healthy

Click Integrations > Load balancing > Edit.

Auto Scaling group: TravelAgencyWebServers

[Details](#) [Integrations](#) [Automatic scaling](#) [Instance management](#) [Instance refresh](#) [Activity](#) [Monitoring](#) [Tags](#)

Load balancing

Load balancer target groups

-

Classic Load Balancers

-

[Edit](#)

Select the relevant load balancer.

Edit TravelAgencyWebServers [Info](#)

Load balancing - optional

Use the options below to attach your Auto Scaling group to an existing load balancer, or to a new load balancer that you define.

Load balancers

☒ Application, Network or Gateway Load Balancer target groups
Only instance target groups that belong to the same VPC as your Auto Scaling group are available for selection.

TravelAgencyWebServer | HTTP

Application Load Balancer: TravelAgencyServers-LB

☐ Classic Load Balancers

Create and attach new load balancers

[Add a new load balancer](#)

[Cancel](#)

[Update](#)

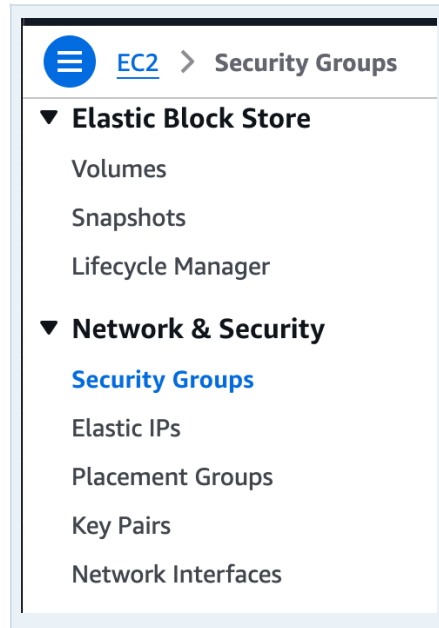
Security Groups

So who is allowed to communicate with our resources?

For the travel agency, we now have:

Internet > ALB > Target Group > EC2 instances

We don't want the EC2 instances to accept web traffic directly from everybody on the internet. We want users to go through the Load Balancer.



Begin by navigating to Security Groups and create a new one.

A screenshot of the AWS Security Groups 'Basic details' form. The form has three sections: 'Security group name' with the value 'TravelAgencyLoadBalancer', 'Description' with the value 'Allow access from internet to the Travel Agency Load balancer', and 'VPC' with the value 'vpc-04e4e39ece7e73d51 (lab/TravelAgencyVpc)'. Each section has an 'Info' link next to it.

With HTTP as the protocol, make it public-facing with 0.0.0.0/0.

A screenshot of the AWS Security Groups 'Inbound rules' form. The form has a table with columns: 'Type', 'Protocol', 'Port range', 'Source', and 'Description - optional'. The first row has the following values: 'HTTP' (with a dropdown arrow), 'TCP', '80', 'Anyw...' (with a dropdown arrow), and '0.0.0.0/0' (with a search icon and a dropdown arrow). There is a 'Delete' button next to the 'Description' field. At the bottom, there is an 'Add rule' button.

Pick the relevant Security Group for the web servers.

The screenshot shows the 'Outbound rules' configuration page in the AWS console. It features a table with columns for 'Type', 'Protocol', 'Port range', 'Destination', and 'Description - optional'. The 'Type' column has a dropdown menu set to 'HTTP'. The 'Protocol' column has a button labeled 'TCP'. The 'Port range' column has a button labeled '80'. The 'Destination' column has a dropdown menu set to 'Custom' and a search bar containing 'sg-0cf6f285abaec5c'. Below the search bar, a button labeled 'sg-0cf6f285abaec5dc' is visible. There is a 'Delete' button in the 'Description - optional' column. At the bottom left, there is an 'Add rule' button.

NOTE: We're currently configuring the Load Balancer's own Security Group, which acts as a firewall controlling what traffic can enter and leave it. The Load Balancer is allowed to send HTTP traffic on port 80 to resources protected by the Web Server Security Group.

Summarised:

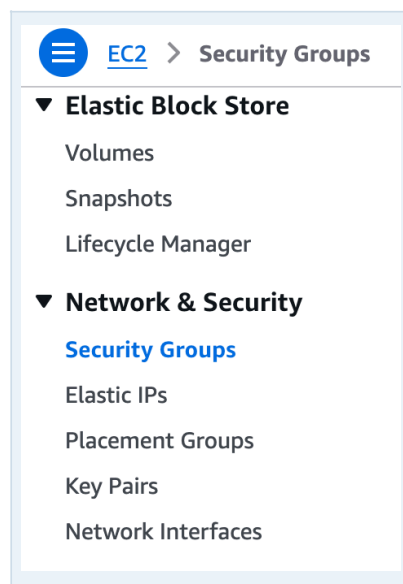
BEFORE: User > Firewall > EC2

AFTER: User > ALB Firewall > ALB > Target Group > EC2 Firewall > EC2

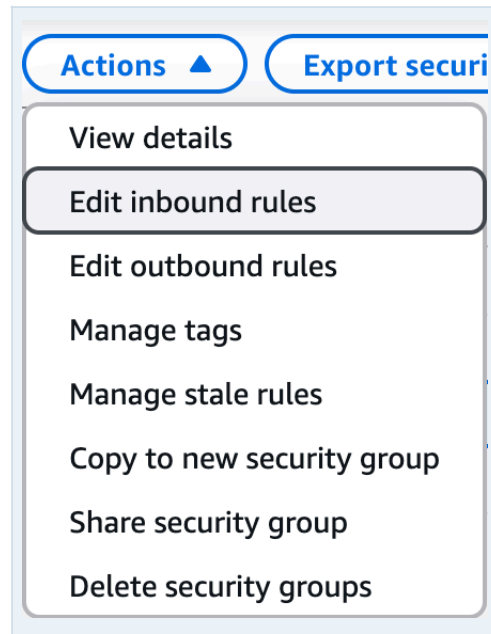
NOTE: This means we also need to configure the Web Server Security Group so that it only accepts traffic from the Load Balancer.

End by reviewing and creating the new Load Balancer Security Group.

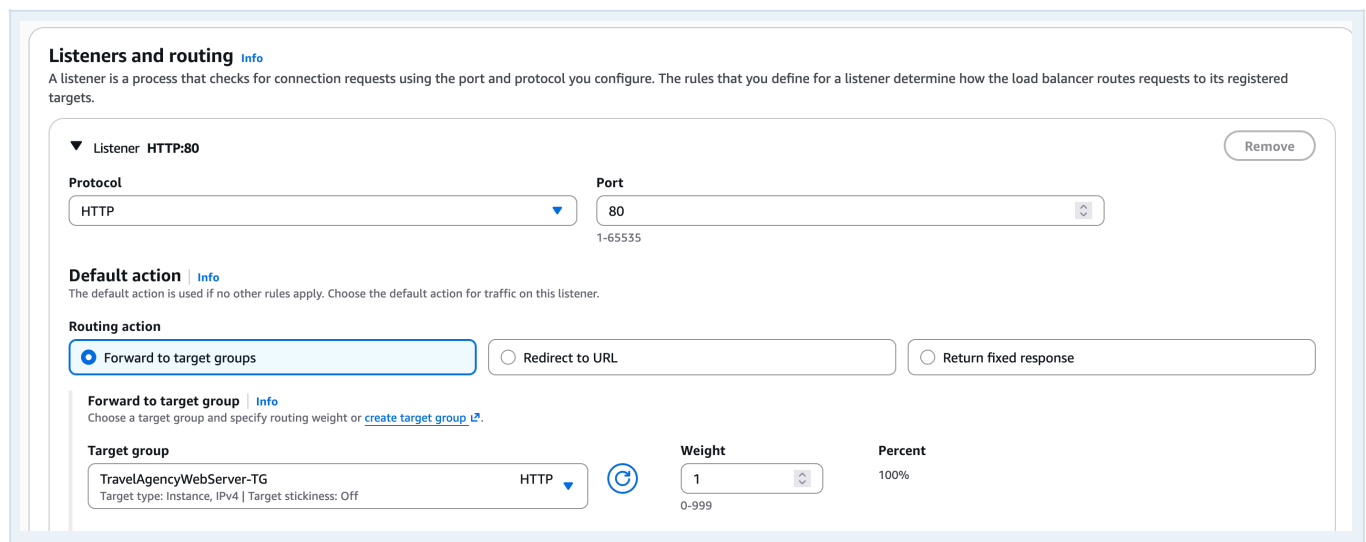
Navigate back to Security Groups and select the WebServer Security Group.



Edit the inbound rules.



Earlier, in the Load Balancer's Listeners and Routing section, we specified the protocol and port. This is what we'll use for the inbound rules.



Finalise and save the rules.

Edit inbound rules

Info

Inbound rules control the incoming traffic that's allowed to reach the instance.

Inbound rules

Info

Security group rule ID	Type	Protocol	Port range	Source	Description - optional	
-	HTTP	TCP	80	Custom	sg-0eeba118fd9fe0f7a	Delete
Add rule						

Cancel

Preview changes

Save rules

Navigate back to the Load Balancer to change its Security Group.

EC2

>

Load balancers

▼ Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

▼ Network & Security

Security Groups

Elastic IPs

Placement Groups

Key Pairs

Network Interfaces

▼ Load Balancing

Load Balancers

Target Groups

Trust Stores

Select the Load Balancer > Security tab > Edit. Remove the old Security Group and replace it with the new Load Balancer Security Group.

Security groups

A security group is a set of firewall rules that control the traffic to your load balancer. Listed security groups are in the same VPC as you selected above. Selected security groups must include at least 1 inbound rule and 1 outbound rule to allow traffic to your load balancer.

Security groups

Select up to 5 security groups

Create security group

TravelAgencyLoadBalancer (sg-0eeba118fd9fe0f7a)

Inbound rule (1) Outbound rule (1)

Cancel

Save changes

AWS Cloud • Part X

Test the Load Balancer and Health Checks

Copy the DNS name and test the connection.

TravelAgencyServers-LB

Actions

▼ Details

Load balancer type

Application

Scheme

Internet-facing

Status

Active

Hosted zone

Z35SXDOTRQ7X7K

VPC

[vpc-04e4e39ece7e73d51](#)

Availability Zones

[subnet-0b27303f5308492ff](#) us-east-1a (use1-az4)

[subnet-0c3bdcad7a7495235](#) us-east-1b (use1-az6)

[subnet-07331938a987e797f](#) us-east-1c (use1-az1)

Load balancer IP address type

IPv4

Date created

Load balancer ARN

[arn:aws:elasticloadbalancing:us-east-1:408687975267:loadbalancer/app/TravelAgencyServers-LB/0a4b37b282fc19b8](#)

DNS name Info

[TravelAgencyServers-LB-1620919492.us-east-1.elb.amazonaws.com](#) (A Record)

You can do a quick in-browser health check by adding /health to the URL.

← → ↺

⛔ Not Secure

http://travelagencyservers-lb-1620919492.us-east-1.elb.amazonaws.com/health

Instance i-07c69f7d126b07aa9 is healthy

NOTE: The default load balancer health check validates only the HTTP server's root path. For more thorough validation, implement robust application health checks to verify server configuration and external access, and monitor the load balancer to confirm active health checks.

Navigate back to Target Groups > select TravelAgencyWebServer > Health checks tab > Edit.

Target groups (1/1) [Info](#) | [What's new?](#)

Filter target groups

< 1 > ⚙

☒	Name	ARN	Port	Protocol	Target type	Load balancer
☒	TravelAgencyWebServer	arn:aws:elasticloadbalancin...	80	HTTP	Instance	TravelAgencyServers-LB

Target group: TravelAgencyWebServer

Details

Targets

Monitoring

Health checks

Attributes

Tags

Health check settings

Edit

Edit the health check path to /health.

Health check path

The port the load balancer uses when performing health checks on targets. By default, the health check port is the same as the target group's traffic port. However, you can specify a different port as an override.

/health

Up to 1024 characters allowed.

Scrolling down, we'll see the default settings mark instances unhealthy after 150 seconds: 5 checks at 30-second intervals.

Health check port

The port the load balancer uses when performing health checks on targets. By default, the health check port is the same as the target group's traffic port. However, you can specify a different port as an override.

☒ Traffic port

☐ Override

Healthy threshold

The number of consecutive health checks successes required before considering an unhealthy target healthy.

5

2-10

Unhealthy threshold

The number of consecutive health check failures required before considering a target unhealthy.

2

2-10

Timeout

The amount of time, in seconds, during which no response means a failed health check.

2

seconds

2-120

Interval

The approximate amount of time between health checks of an individual target

5

seconds

5-300

NOTE: The new settings mark instances unhealthy after 10 seconds: 2 checks at 5-second intervals.

Save the changes.

Auto Scaling Groups - Private Subnets (Optional)

After creating an internet-facing load balancer, you can run your application in private subnets. Add or remove subnets associated with the Auto Scaling Group as needed.

Navigate back to Auto Scaling Groups and select the relevant group.

Auto Scaling groups (1/1) Info

Last updated 1 minute ago

[Launch configurations](#) [Launch templates](#) [Actions](#) [Create Auto Scaling group](#)

Search your Auto Scaling groups

Name	Status - new	Launch template/configuration	Instances	Instance health - new
TravelAgencyWebServers	At desired capacity	TravelAgencyTemplate-7f44f7c0 Version 1		1/1 Healthy

Auto Scaling group: TravelAgencyWebServers

Network [Edit](#)

Availability Zones	Subnet ID	Availability Zone distribution
use1-az4 (us-east-1a)	subnet-0b27303f5308492ff	Balanced best effort

In a real setup, for a highly available web application behind an ALB, I would select multiple subnets across multiple Availability Zones here.

Network

For most applications, you can use multiple Availability Zones and let EC2 Auto Scaling balance your instances across the zones. The default VPC and default subnets are suitable for getting started quickly.

Availability Zones and subnets

Define which Availability Zones and subnets your Auto Scaling group can use in the chosen VPC.

Select Availability Zones and subnets

us-east-1a | subnet-0b27303f5308492ff (lab/TravelAgencyVpc/PublicSubnet1)
10.0.0.0/19

[Create a subnet](#)

Availability Zone distribution - new

Auto Scaling automatically balances instances across Availability Zones. If launch failures occur in a zone, select a strategy.

☒ **Balanced best effort**

If launches fail in one Availability Zone, Auto Scaling will attempt to launch in another healthy Availability Zone.

☐ **Balanced only**

If launches fail in one Availability Zone, Auto Scaling will continue to attempt to launch in the unhealthy Availability Zone to preserve balanced distribution.

☐ **Reservations Then Balanced**

Prioritizes launching into Capacity Reservations, distributing across AZs with available reservations. When reservations are fully utilized, falls back to balanced on-demand.

[Cancel](#) [Update](#)

So instead of only:

us-east-1a > PublicSubnet1

I would configure the Auto Scaling Group with something like:

- us-east-1a > PrivateSubnetA
- us-east-1b > PrivateSubnetB
- us-east-1c > PrivateSubnetC

Why private subnets? Because the ALB is the public entry point. The EC2 instances do not need to be directly reachable from the internet.

Auto Scaling Groups - Is It Working? (Optional)

Let's demonstrate whether an instance gets created after our desired-capacity changes.

The screenshot shows the AWS Management Console 'Instances' page with 1 instance. The instance is named 'TravelAgency...', has ID 'i-07c69f7d126b07aa9', is in a 'Running' state, and is of type 't3.micro'. A context menu is open over the instance, showing options: Stop instance, Start instance, Reboot instance, Hibernate instance, and Terminate (delete) instance. The 'Terminate (delete) instance' option is highlighted.

	Name	Instance ID	Instance state	Instance type	Availability Zone	Public IPv4 DNS
☒	TravelAgency...	i-07c69f7d126b07aa9	Running	t3.micro	us-east-1a	ec2-52-203-230-11

Refresh and wait up to a minute.

The screenshot shows the AWS Management Console 'Instances' page with 2 instances. A green notification banner at the top states 'Successfully initiated termination (deletion) of i-07c69f7d126b07aa9'. The first instance, 'TravelAgency...' with ID 'i-07c69f7d126b07aa9', is now in a 'Terminated' state. The second instance, 'TravelAgency...' with ID 'i-0a944f8c255c4bfef', is in a 'Running' state. The 'Status check' column shows '-' for the terminated instance and 'Initializing' for the running instance.

	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS
☐	TravelAgency...	i-07c69f7d126b07aa9	Terminated	t3.micro	-	us-east-1a	-
☐	TravelAgency...	i-0a944f8c255c4bfef	Running	t3.micro	Initializing	us-east-1a	ec2-3-84-14

Add Another Availability Zone

Let's finalise by changing the Availability Zones.

Auto Scaling Groups > Network > Edit.

Availability Zones and subnets
Define which Availability Zones and subnets your Auto Scaling group can use in the chosen VPC.

Select Availability Zones and subnets

us-east-1a | subnet-0b27303f5308492ff (lab/TravelAgencyVpc/PublicSubnet1)
10.0.0.0/19

us-east-1b | subnet-0c3bdcad7a7495235 (lab/TravelAgencyVpc/PublicSubnet2)
10.0.32.0/19

Create a subnet

Availability Zone distribution - new
Auto Scaling automatically balances instances across Availability Zones. If launch failures occur in a zone, select a strategy.

☒ **Balanced best effort**
If launches fail in one Availability Zone, Auto Scaling will attempt to launch in another healthy Availability Zone.

☐ **Balanced only**
If launches fail in one Availability Zone, Auto Scaling will continue to attempt to launch in the unhealthy Availability Zone to preserve balanced distribution.

☐ **Reservations Then Balanced**
Prioritizes launching into Capacity Reservations, distributing across AZs with available reservations. When reservations are fully utilized, falls back to balanced on-demand.

Cancel

Update

After reviewing the successful change, go back to the selected Auto Scaling Group and edit it.

Auto Scaling group: TravelAgencyWebServers

Details

Integrations

Automatic scaling

Instance management

Instance refresh

Activity

Monitoring

Tags

TravelAgencyWebServers Capacity overview

arn:aws:autoscaling:us-east-1:408687975267:autoScalingGroup:cbe85b27-03d8-46b5-887b-02388917d75a:autoScalingGroupName/TravelAgencyWebServers

Desired capacity 1	Scaling limits 1 - 1	Desired capacity type Units (number of instances)	Status ✔ At desired capacity
------------------------------	--------------------------------	---	--

Since we now have two Availability Zones, we can now have one EC2 instance in each.

Verify by going back to EC2 instances.

×

Group size

Specify the size of the Auto Scaling group by changing the desired capacity. You can also specify minimum and maximum scaling limits.

Desired capacity type

Choose the unit of measurement for the desired capacity value. vCPUs and Memory(GiB) are only supported for mixed instances groups configured with a set of instance attributes.

Units (number of instances) ▼

Desired capacity

Specify your group size.

2

Scaling limits

Set limits on how much your desired capacity can be increased or decreased.

Min desired capacity

1

Equal or less than desired capacity

Max desired capacity

2

Equal or greater than desired capacity

Cancel

Update

Summary

The travel agency application was redesigned to improve availability, scalability, and security.

Route 53 directs users to the application, while CloudFront can cache and deliver content closer to users. An Application Load Balancer receives web traffic and forwards requests through a Target Group to healthy EC2 instances.

The EC2 instances are managed by an Auto Scaling Group, which maintains the desired number of instances, replaces failed instances, and can distribute them across multiple Availability Zones. CloudWatch provides metrics that can be used to automatically scale capacity when demand changes.

Security Groups control traffic between each layer. The Load Balancer Security Group accepts public web traffic, while the Web Server Security Group can restrict EC2 traffic so requests come through the load balancer rather than directly from the internet.